

产品需求与设计文档3.0

[居家智能摄像头监控]
[home-camera-monitor]

产品需求与设计文档
Version: 3.0

[组名：大组长小组员]

1. 版本说明

在每次需求设计评审之后，必然会产生相应的内容调整，对每次版本更新做简单记录。

更新时间	更新人	更新内容简介
2026.7.06	刘澎湖	确定项目方向和大致内容（居家智能摄像头监）
2026.7.08	刘澎湖	完成v1.0需求文档(项目背景与意义、项目目标、需求分析)
2026.7.11	刘澎湖	完成 v2.0需求文档（架构设计、详细设计、接口设计）
2026.7.12	刘澎湖	新增功能 修改完善部分功能
2026.7.13	刘澎湖	所有图片绘制完成
2026.7.14	刘澎湖	完成v3.0需求文档，全部更新修改完成

2. 产品概述

2.1 项目背景与意义

当前，智能家居安防需求日益增长，用户希望通过家庭摄像头实时监控家中情况，保障家人安全。传统监控系统仅具备录像功能，无法实现智能分析和主动告警。随着计算机视觉和深度学习技术的发展，基于AI的智能监控系统能够自动识别人脸、检测异常行为、识别危险情况，大幅提升家庭安全防护能力。

本项目旨在构建一套居家智能摄像头监控系统，实现人脸识别（区分家人与陌生人）、危险区域管控（如厨房禁止小孩进入）、异常事件检测（着火、摔倒）等核心功能，为用户提供全方位的智能安防解决方案。

2.2 项目目标（项目主要工作内容）

目标	说明
家庭成员识别	注册家庭成员人脸，实时识别并标注身份，活体反欺骗（多帧启发式）

人员统计	统计当前画面/家中人数，区分熟人与陌生人
陌生人告警	未注册人脸出现即触发告警
危险区域	可配置禁区（如厨房禁止小孩进入），闯入即告警
异常检测	着火、人员摔倒等紧急情况识别（≥2 种）异常声学
告警管理	统一告警展示、处置、日志、事件回放、钉钉通知 和AI 监控日报

2.3 技术方案

系统采用前后端分离架构：Vue3 前端负责监控展示与交互，Django + DRF 后端承载业务 API 与数据持久化，AI 分析模块（OpenCV / dlib / YOLOv8）与视频拉流同进程集成。摄像头/OBS 经 RTMP 推流至 MediaMTX，浏览器以 WebRTC 低延迟预览原始画面，前端 Canvas 轮询

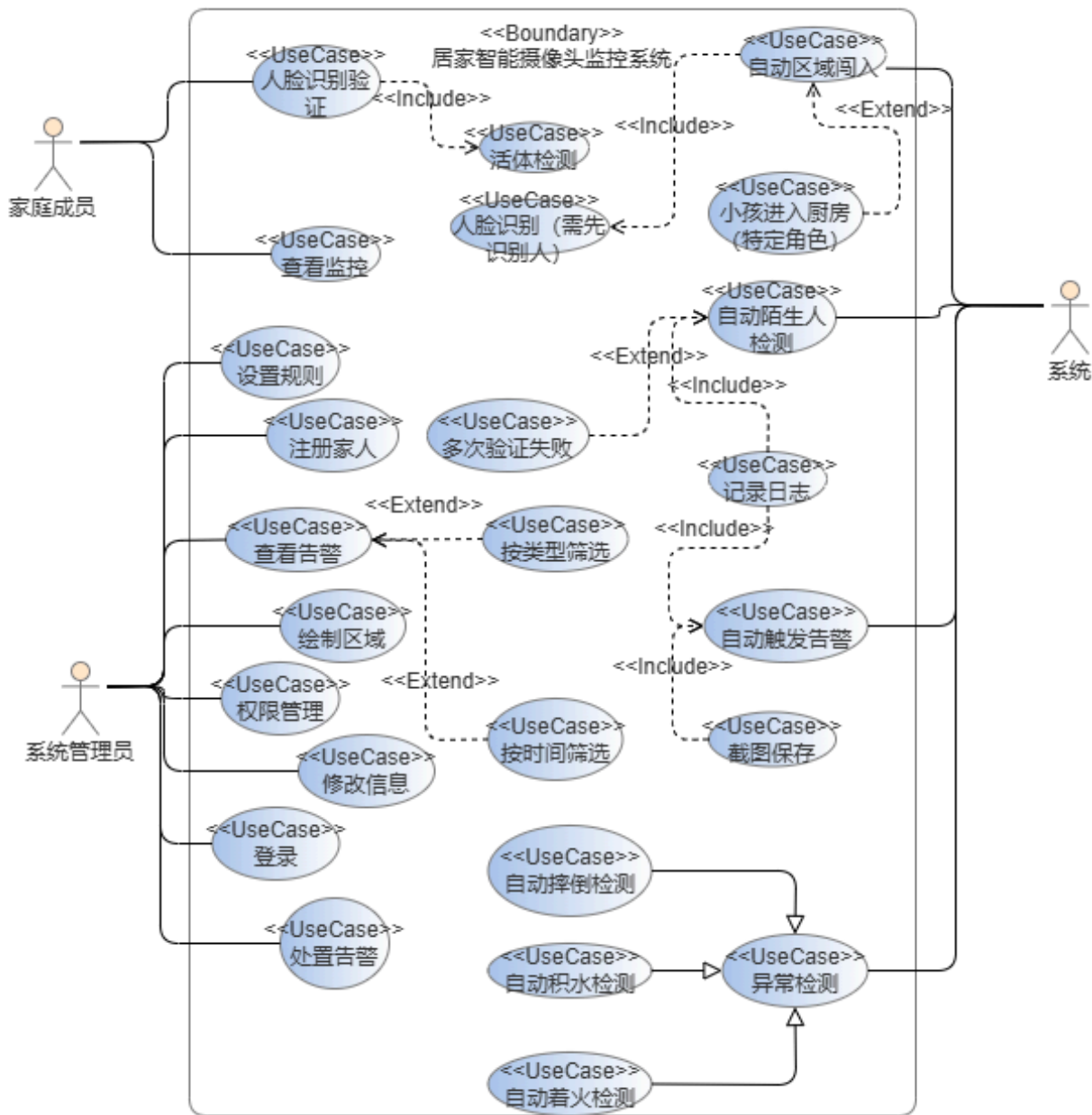
`/api/video/presence/` 叠加 AI 标注； `/video_feed/` MJPEG 仅作调试备用。

层次	技术选型	说明
前端	Vue3 + Vite + Element Plus	监控页 WebRTC iframe + FaceOverlay Canvas； ZoneEditor 同构
业务后端	Django 4.2 + DRF + SimpleJWT	业务 API + Swagger； AI 与业务同进程
人脸识别	dlib + face_recognition	128 维编码；多帧活体注册与启发式反欺骗
目标检测	YOLOv8n（HOG 降级）	行人检测 + 禁区多边形
异常检测	OpenCV 颜色/形态 + PANNs 音频	着火 / 摔倒；声学
数据库	MySQL	用户、家庭、区域、告警、事件、日报
流媒体	OBS → RTMP(:9090) → MediaMTX → RTSP(:8554)	WebRTC(:8889) 主预览； MJPEG /video_feed/ 备用
AI 展示	<code>/api/video/presence/</code>	前端轮询叠加框数据
部署	gunicorn(:8010) + Nginx 反代	生产单体后端

3. 需求分析

3.1 系统需求总体概述

- 家庭成员：被监控对象，通过人脸识别被系统识别 | 权限 被识别、触发告警
- 系统管理员：系统使用者，管理家人信息和监控配置 | 权限 注册家人、设置区域、查看告警、处置事件
- 系统（自动）：自动执行检测和告警逻辑 | 权限：自动识别、自动告警、自动记录



用例描述：

- 系统总体用例包含六大核心模块：
- 视频监控模块：用户可实时查看客厅、厨房等多路监控画面，支持历史事件回放。
- 人脸识别模块：管理员可注册家人人脸信息（含角色），系统自动识别家人/陌生人，实时统计人数。
- 区域管理模块：管理员通过Canvas绘制危险区域，设置禁区规则，系统自动检测区域闯入。
- 异常检测模块：系统自动检测着火、摔倒、异常声学等异常情况并触发告警。

告警中心模块：管理员查看告警列表、处置告警、查询历史告警日志。

用户管理模块：管理员登录系统、管理家人信息、配置系统参数

3.2 功能性需求分析

3.2.1 人脸识别模块

功能描述：基于dlib实现人脸检测与128维特征编码比对，支持家人注册、实时识别、陌生人告警与人数统计。

用例ID	用例名称	参与者	前置条件	后置条件
UC-001	注册家人人脸	管理员	已登录系统	人脸库新增记录
UC-002	实时人脸识别	系统	摄像头正常工作	标注识别结果
UC-003	陌生人检测	系统	人脸库已初始化	触发FACE_UNKNOWN告警
UC-004	人数统计	系统	视频流正常	更新人员统计数据
UC-005	人脸库维护（更新/删除家人人脸）	管理员	已登录系统，该家人已注册	人脸库记录更新/删除，关联告警保留
UC-006	活体反欺骗检测	系统	摄像头检测到人脸	判定是否为真实人脸，拒绝假体攻击
UC-007	查看人脸库成员	管理员	已登录系统，人脸库已初始化	展示已注册家人列表

业务流程：

1.家人注册

管理员录入家人信息（姓名、角色 adult/child）

主路径：多帧活体采集（≥3 帧）注册；单图上传可保留为弱化说明

后端提取 128 维特征编码，存入 MySQL 人脸库

2.实时识别

视频流每帧提取人脸特征

与注册库进行欧氏距离比对（阈值 0.45）

匹配成功：绿框显示姓名和角色

匹配失败：红框标注"陌生人"

3.陌生人告警

陌生人持续出现超过阈值

触发 FACE_UNKNOWN 告警，截图保存，告警写入后，告警中心轮询刷新（约 3s）；

4.人数统计

实时统计帧内人脸数量

区分家人/陌生人，前端HomeMonitor / Overlay：约 200ms 级轮询更新面板

5.人脸库维护

管理员查询/更新/删除已注册家人信息

同步更新人脸库与 MySQL 记录

6.活体反欺骗

「帧间运动 / 回放环路 / 纹理启发式」；告警类型 FACE_SPOOF / FACE_REPLAY / FACE_DEEPFAKE
拒绝照片、视频、面具等假体攻击

3.2.2 区域管控模块

功能描述： 支持在监控画面上通过 Canvas 绘制多边形危险区域，设置禁区规则（如禁止 child 进入厨房），系统自动检测人体位置并判断闯入行为，对违规闯入触发 INTRUSION（另有 PROXIMITY 过近、LOITER 停留）告警。

用例ID	用例名称	参与者	前置条件	后置条件
UC-008	绘制危险区域	管理员	已登录系统	区域配置保存
UC-009	设置禁区规则	管理员	区域已绘制	规则生效
UC-010	区域闯入检测	系统	区域和规则已配置	触发INTRUSION告警
UC-011	查询危险区域	管理员	已登录系统	展示区域列表及多边形坐标
UC-012	修改危险区域	管理员	已登录系统，区域已存在	区域坐标/规则更新
UC-013	删除危险区域	管理员	已登录系统，区域已存在	区域配置删除，关联规则失效

业务流程：

1. 区域配置

管理员在 Canvas 上绘制多边形区域（如厨房）

设置区域名称、关联视频流、禁止角色（如 child）

保存至 MySQL，异步推送至检测服务

支持配置安全距离 `safe_distance`、停留时长 `dwell_time`，分别触发 `PROXIMITY`、`LOITER` 告警

2. 区域管理

查询已配置区域列表及坐标

修改顶点坐标或删除区域配置

3. 闯入检测

YOLOv8n 优先，HOG 降级检测人体位置，计算中心点坐标

射线法判断是否在禁区多边形内

结合人脸识别获取人员角色

若角色匹配 forbidden_roles，触发 INTRUSION 告警

3.2.3 异常检测模块

功能描述：基于图像分析实现着火、摔倒两种异常检测，同时支持音频异常检测与音视频联动。通过 HSV 色彩分析、高亮区域阈值判断、人体框高宽比计算等算法，自动识别紧急情况并触发告警。

用例ID	用例名称	参与者	前置条件	后置条件
UC-015	着火检测	系统	高亮红/黄区域阈值判断	FIRE
UC-016	摔倒检测	系统	人体框高宽比低于阈值	FALL
UC-017	音频异常检测	系统	音频采集设备正常	触发 SCREAM/CRYING/GLASS_BREAK 等告警
UC-018	检测灵敏度配置	管理员	已登录且为 superuser	检测阈值更新，新配置即时生效
UC-019	音视频联动检测	系统	视频与音频同时触发异常	提升告警优先级为 EMERGENCY

业务流程：

1. 灵敏度配置

superuser 调整检测阈值

新配置即时生效

2. 着火检测

检测高亮红/黄火焰颜色区域

超面积阈值则触发 FIRE 紧急告警

3. 摔倒检测

YOLOv8n 优先，HOG 降级检测人体，计算框高宽比

低于躺倒阈值则触发 FALL 告警

4. 音频异常检测

分析环境音频特征，识别哭声、玻璃破碎、呼救声

触发对应音频类告警

5. 音视频联动

视频与音频同时触发异常时

告警优先级提升为 EMERGENCY

3.2.4 视频监控模块

功能描述：基于 MediaMTX（原 Nginx-RTMP）实现视频流推流与拉流，支持多路摄像头实时画面展示、视角切换、手动抓拍与历史回放。后端通过 OpenCV 拉取 RTSP 流，经 AI 处理链叠加标注后，以浏览器 WebRTC（:8889）播原始流 + Canvas 叠 AI 框推送至前端无插件播放。

用例ID	用例名称	参与者	前置条件	后置条件
UC-020	实时视频查看	管理员	摄像头已配置并推流	WebRTC 画面 + Canvas AI 标注
UC-021	多摄像头切换	管理员	存在多路摄像头配置	切换至指定摄像头画面
UC-022	视频流拉流	系统	RTMP 服务正常运行	RTSP 拉流 + process_frame； MJPEG /video_feed/ 为调试/备用
UC-023	历史回放	管理员	告警/事件已关联截图	展示历史截图或片段
UC-024	手动抓拍截图	管理员	视频流正常，已登录	截图保存至 /static/snapshots/
UC-025	多路分屏查看	管理员	存在多路摄像头配置且均推流正常	前端同时展示多路实时画面
UC-026	告警关联回放	管理员	告警已关联视频片段	播放约 10 秒 MP4 短视频（快照 + clip）

业务流程：

1. 实时查看

- 1. 管理员打开 HomeMonitor，选择客厅/厨房摄像头
- 2. 前端以 WebRTC iframe 加载 `http://{IP}:8889/stream/{id}/` 显示原始画面
- 3. 前端轮询 `GET /api/video/presence/?stream_id={id}`，Canvas (FaceOverlay) 叠加人脸/告警框
- 4. 后端从 MediaMTX 拉 RTSP，执行 face + detection 处理链，并发布 presence
- 5. `/video_feed/{id}` MJPEG 仅作调试/备用，不作为监控页主路径

2. 摄像头切换

点击切换按钮，切换 stream_id / WebRTC 地址 + presence 分路
后端释放原资源，切换新源重新拉流

3. 多路分屏

前端同时请求多路视频流
后端多线程分别处理，前端分屏展示

4. 手动抓拍

管理员点击抓拍按钮，截取当前帧保存至 `/static/snapshots/`

5. 历史回放

根据告警关联的 snapshot_path 或视频片段
展示告警触发时截图，支持快照 + 约 10 秒 MP4 短视频视频回放

3.2.5 告警中心模块

功能描述：统一接收人脸识别、区域管控、异常检测三大模块触发的告警，提供告警展示、筛选、处置、日志查询、截图回放、通知推送与告警升级功能。支持按类型、状态、时间范围筛选，形成告警全生命周期闭环管理。

用例ID	用例名称	参与者	前置条件	后置条件
UC-027	查看告警列表	管理员	系统已产生告警记录	展示分页告警列表
UC-028	按类型筛选告警	管理员	告警列表已加载	显示筛选后结果
UC-029	处置告警	管理员	告警状态为 pending	告警状态更新为 handled
UC-030	告警截图回放	管理员	告警已关联 snapshot_path	展示告警触发时截图
UC-031	查看告警日志	管理员	事件已记录	展示时间线事件记录
UC-032	告警通知配置	管理员	已登录且为 superuser	钉钉 Webhook 配置保存

UC-033	告警升级链	系统	告警状态为 pending 且超过设定时间	告警级别提升，通知范围扩大
UC-034	忽略告警	管理员	告警状态为 pending	告警状态更新为 ignored
UC-035	导出ai告警日志	管理员	事件已记录	生成下载 Markdown

业务流程：

1. 告警触发

人脸识别/区域管控/异常检测/音频检测触发异常

统一调用 create_alert() 写入 MySQL 告警表

2. 实时推送

列表轮询刷新

更新告警列表实时展示

3. 查看与筛选

按类型（陌生人/闯入/着火/摔倒）、状态（pending/handled/ignored）、时间范围筛选

4. 告警处置

点击"处置"：状态更新为 handled，记录处理时间

点击"忽略"：状态更新为 ignored

5. 告警升级

pending 状态超过设定时间未处理

自动提升告警级别，扩大通知范围（钉钉→钉钉 Webhook + 超时升级）

6. 通知配置

superuser 配置钉钉 Webhook 地址

高优先级告警自动推送至指定群

7. 日志与回放

EventLog 展示时间线事件记录

支持按日期查询，导出 Excel/PDF 报表

根据 snapshot_path 回放告警截图

3.2.6 用户管理模块

功能描述：提供基于 JWT 的用户认证与授权，支持管理员手机号注册/登录、家人信息 CRUD、角色权限控制（admin/member）及系统基础设置。采用 Django 密码哈希与 SimpleJWT 无状态认证，支

持 Token 刷新与过期失效机制。

表格用例 ID	用例名称	参与者	前置条件	后置条件
UC-036	用户登录	管理员	账号已注册	签发 JWT Token，进入系统
UC-037	用户注册	管理员	手机号未注册	创建新用户账号
UC-038	家人信息管理	管理员	已登录系统	新增/修改/删除家人信息
UC-039	权限控制	系统	用户已登录	校验操作权限
UC-040	系统设置	管理员	已登录且为 superuser	更新系统配置
UC-041	修改密码	管理员	已登录系统，原密码校验通过	密码哈希更新，原 Token 失效
UC-042	找回密码	管理员	手机号已注册，短信验证码通过	密码重置成功
UC-043	退出登录	管理员	已登录系统	Token 清除，返回登录页
UC-044	批量导入家人	管理员	已登录系统，上传文件格式正确	批量创建家人记录
UC-045	摄像头参数配置	管理员	已登录且为 superuser	RTSP 地址、分辨率等参数更新

业务流程：

1. 用户注册

输入手机号，校验未注册

Django 生成密码哈希，创建账号

2. 用户登录

输入手机号和密码，后端 Django 密码校验

验证通过生成 JWT Token（access + refresh）

前端存储 Token，后续请求自动注入 Authorization 头部

3. 密码管理

修改密码：原密码校验通过后更新，原 Token 失效

找回密码：手机号验证+短信验证码通过后重置

4. 退出登录

清除前端 Token，返回登录页

5. 家人信息管理

录入家人信息（姓名、角色 adult/child/elder/guest）

支持单条增删改查及批量导入导出

6. 权限控制

通过 role 字段（admin/member）校验权限

非 admin 无法进入系统设置与区域编辑

多家庭隔离：携带 X-Active-Household-Id 请求头，数据完全隔离

7. 系统设置

仅 superuser 可配置网络参数、摄像头 RTSP 地址、告警阈值、钉钉通知地址

配置即时生效

4. 概要设计

4.1 系统架构设计

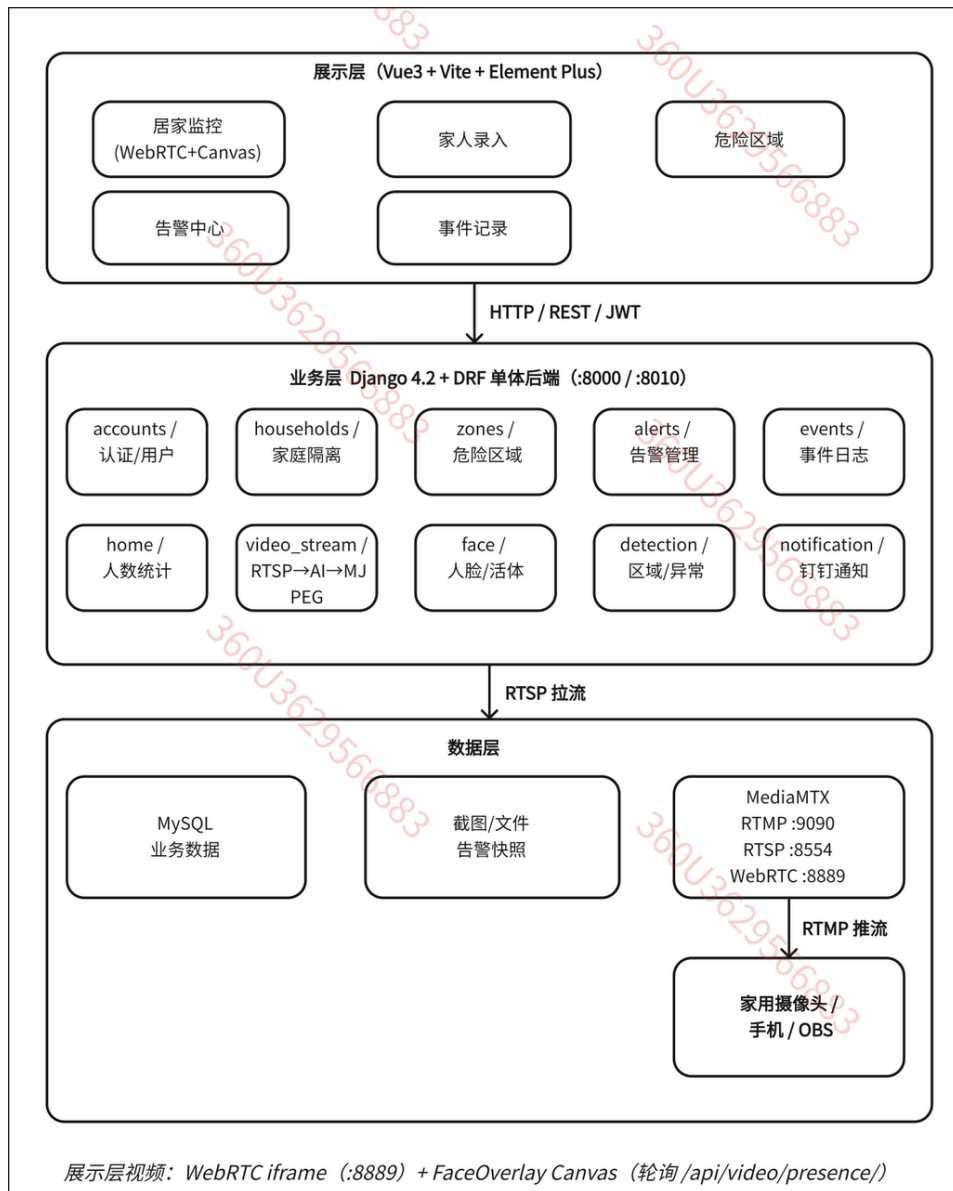


图4-1 系统架构图

4.2 系统功能模块设计

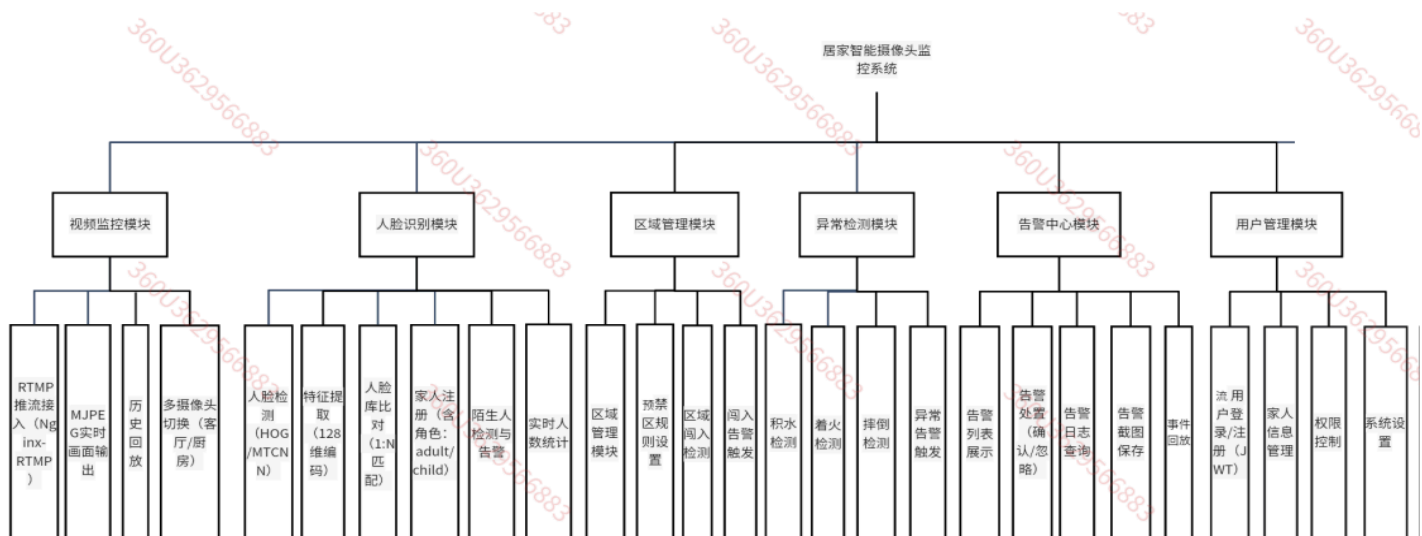


图4-2 系统功能结构图

4.1.1 用户管理模块

模块定位：系统的身份认证与权限管理基础，负责用户注册登录、家庭关系维护及系统级配置。

A accounts（认证/用户）

手机号注册/登录：提供基于手机号码的用户注册与登录接口，支持短信验证码校验。

JWT 认证：采用 JSON Web Token 实现无状态身份认证，登录成功后返回 Token，后续请求携带 Token 进行权限校验，支持 Token 刷新与过期失效机制。

家庭成员 CRUD：管理家庭成员的基本信息（姓名、年龄、关系、联系方式等），支持增删改查与批量导入导出，关联人脸识别模块为家人绑定人脸数据与角色标签（adult/child）。

B households（家庭隔离）

家庭创建/加入/审核：支持用户创建家庭组，其他成员通过邀请码或链接申请加入，家庭管理员审核通过后生效。

摄像头管理：对家庭内的摄像头设备进行统一管理，包括设备添加、删除、状态监控与参数配置。

多家庭隔离：实现不同家庭之间的数据完全隔离，确保 A 家庭用户无法访问 B 家庭的视频、告警等敏感数据，保障隐私安全。

C reports（AI 监控日报）

AI 监控日报生成：每日自动生成家庭安全监控日报，汇总当日的人脸识别记录、异常检测事件、告警处置情况等数据，以图表形式展示安全态势。

4.2.2 视频监控模块

模块定位：系统的视频数据采集与实时展示核心，负责摄像头接入、视频流转码与画面输出。

video_stream（视频流处理）

RTSP 拉流：从 MediaMTX 流媒体服务器或家用摄像头拉取 RTSP 视频流，建立稳定的视频传输通道。

AI 处理链：将拉取的视频帧送入人脸识别、异常检测等 AI 算法模块进行实时分析，提取人脸特征、检测人体姿态、识别危险区域闯入等。

Overlay API：在视频画面上叠加 AI 分析结果（如人脸框、区域边界线、异常标记），生成带有标注信息的实时画面，供前端展示。

MJPEG 备用输出：当 WebRTC 通道不稳定时，提供 MJPEG 格式的备用视频流输出，确保浏览器端始终能查看实时画面。

4.2.3 人脸识别模块

模块定位：系统的智能身份识别核心，基于计算机视觉技术实现家庭成员与陌生人的自动区分，并提供人数统计能力。

A face（人脸/活体）

人脸注册：提供家人信息录入接口，支持上传照片或实时抓拍进行人脸注册，提取 128 维特征向量存储于数据库，为每位家人分配角色标签（成人 adult / 儿童 child）。

人脸检测与识别：采用 dlib 算法从视频帧中定位人脸区域，提取特征后与数据库中已注册家人特征进行 1:N 匹配，采用欧氏距离或余弦相似度计算相似度，设定阈值判断是否匹配成功。

活体反欺骗：通过启发式活体信息分析，防止照片、视频、面具等假体攻击，确保识别结果真实可靠。

陌生人告警：当检测到未注册人脸（匹配失败）时，自动标记为"陌生人"，触发告警机制，向用户推送陌生人闯入通知，并保存现场截图作为证据。

B home（人数统计）

人数统计 API：读取 face 服务实时分析快照，统计当前画面中出现的人数，支持按区域分别统计，生成人数变化趋势图供前端展示。

4.2.4 区域管理模块

模块定位：系统的空间安全管控核心，实现居家区域划分、危险区域标记与闯入检测。

zones（危险区域）

危险区域 CRUD：提供可视化区域划分工具的后端接口，支持在监控画面上绘制多边形区域（如客厅、厨房、阳台），并对区域信息进行增删改查与持久化存储。

forbidden_roles 配置：为每个危险区域设置禁止进入的角色列表（如阳台禁止 child 进入），实现差异化的区域管控策略。

区域闯入检测：实时分析人体位置坐标，判断是否进入预设的危险区域，结合人脸识别结果区分闯入者身份（成人/儿童），对儿童闯入危险区域触发高优先级告警。

4.2.5 异常检测模块

包含子模块：

detection（区域/异常）

禁区闯入：在非允许时段（如深夜）检测到人员活动时，结合人脸识别结果区分家人正常活动与陌生人非法闯入，触发相应告警。

着火检测：基于火焰颜色特征与运动模式识别算法，检测画面中的明火或烟雾，检测到火情时立即触发最高级别告警，并推送紧急通知。

跌倒检测：基于行人检测框的宽高比（躺倒形态时宽高比异常）判定摔倒，持续满足阈值则触发 FALL 告警；行人检测优先 YOLOv8n，不可用时降级为 HOG。

音频异常：分析环境音频特征，识别玻璃破碎、婴儿哭声、呼救声等异常声音，与视频检测联动提升告警准确率。

音视频联动：当视频检测与音频检测同时触发异常时，提升告警优先级，避免单一模态误报。

4.2.6 告警中心模块

模块定位：系统的告警事件管理与处置中心，负责告警的全生命周期管理、通知推送与事件回溯。

A alerts（告警管理）

告警管理：以列表形式展示所有告警事件，支持按时间、类型、状态、优先级筛选，列表显示告警缩略图、发生时间、告警类型、处理状态等关键信息。

create_alert 服务层：统一封装告警创建逻辑，接收来自异常检测、人脸识别、区域管理等模块的异常事件，生成标准化告警记录，自动分配告警优先级（低/中/高）。

告警处置（确认/忽略）：用户可对告警进行"确认"（已知晓并处理）或"忽略"（误报）操作，处置操作记录操作人、处置时间和处置意见，形成闭环管理。

B events（事件日志）

事件日志：记录系统运行过程中的所有关键事件（包括告警触发、用户操作、系统异常等），提供全文检索与高级筛选功能，支持按时间段导出事件日志报表（Excel/PDF 格式）。

事件回放：支持从告警时间点约 10 秒进行视频回放，帮助用户还原事件经过，辅助判断告警真伪与事件原因。

C notifications（钉钉通知）

钉钉 Webhook 推送：集成钉钉机器人 Webhook，当高优先级告警触发时，自动向指定钉钉群或用户推送告警消息，包含告警类型、发生时间、现场截图等关键信息。

告警升级链：当告警在设定时间内未被处理时，自动升级告警级别并扩大通知范围（如从普通用户通知升级到管理员通知，从钉钉推送到短信/电话通知），确保紧急事件得到及时响应。

4.3数据库设计

accounts_user — 用户

字段	类型	说明
id	BIGINT PK	主键
phone	VARCHAR(16) UNIQUE	手机号（登录凭证）
password	VARCHAR	密码哈希

household — 家庭

字段	类型	说明
id	BIGINT PK	主键

name	VARCHAR(64)	家庭名称
created_by	FK → User	创建者

family_member — 家庭成员

字段	类型	说明
id	BIGINT PK	主键
household_id	FK → Household	所属家庭
name	VARCHAR(64)	姓名
role	VARCHAR(16)	adult / child / elder / guest
student_id	VARCHAR(32)	学号/工号（可选）
face_encoding	JSON	128 维特征向量
is_active	BOOL	是否启用

zone — 危险区域

字段	类型	说明
id	BIGINT PK	主键
household_id	FK → Household	所属家庭
name	VARCHAR(64)	如「厨房」
stream_id	VARCHAR(32)	living_room / kitchen
points_json	JSON	多边形坐标
forbidden_roles	JSON	禁止进入的角色，如 ["child"]
is_active	BOOL	是否启用

alert — 告警

type	含义
FACE_UNKNOWN	陌生人

FACE_SPOOF / FACE_REPLAY / FACE_DEEPFAKE	活体反欺骗攻击
INTRUSION	危险区域闯入
PROXIMITY	距离过近
LOITER	异常停留
FIRE	着火
FALL	摔倒
SCREAM / FIGHT / CRYING / GLASS_BREAK	音频异常
EMERGENCY	音视频联动紧急事件

event_log — 事件日志

字段	类型	说明
household_id	FK → Household	所属家庭
event_type	VARCHAR(32)	事件类型
stream_id	VARCHAR(32)	视频流 ID
description	VARCHAR(256)	事件描述
metadata	JSON	附加数据
created_at	DATETIME	发生时间

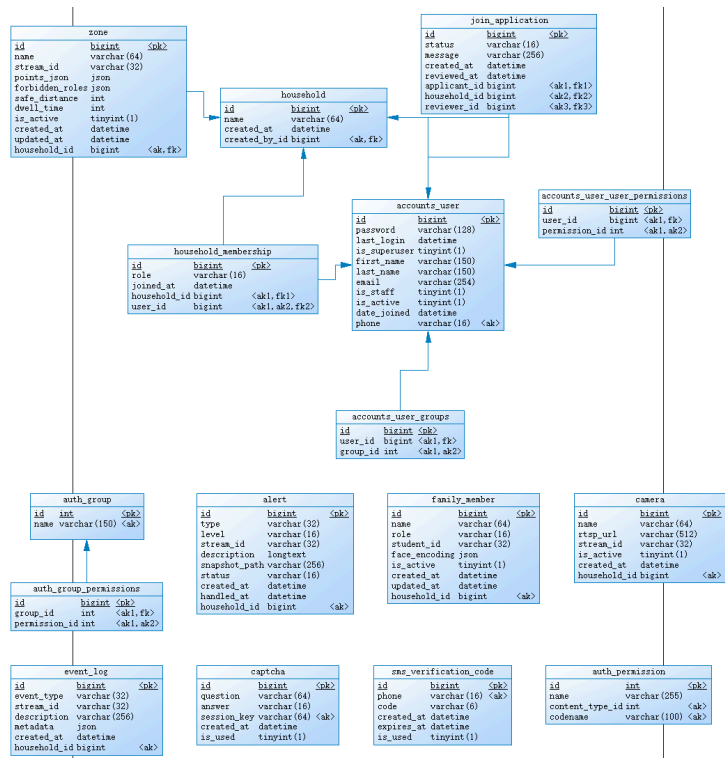


图4-3 系统实体关系图

数据库表清单（共 20 张）		
一、业务表（11 张，需要画 ER 图的核心表）		
表名	说明	主要字段
accounts_user	用户表（手机号登录，无用户名）	id, phone(手机号/UNI), password, is_superuser, is_staff, is_active, last_login, date_joined
household	家庭/住户表	id, name(家庭名称), created_by_id(→accounts_user, 创建者), created_at
household_membership	用户-家庭 多对多关系表	id, household_id(→household), user_id(→accounts_user, role(admin/member), joined_at
join_application	加入家庭申请表	id, household_id(→household), applicant_id(→accounts_user, 申请人), reviewer_id(→accounts_user, 审核人), status(pending/approved/rejected), message, created_at, reviewed_at
family_member	家庭成员表（被摄像头识别的人）	id, household_id(→household), name, role(adult/child/elder/guest), student_id, face_encoding(USON, 128维向量), is_active, created_at, updated_at
camera	摄像头表	id, household_id(→household), name, rtsp_url, stream_id, is_active, created_at
zone	危险区域表	id, household_id(→household), name, stream_id, points_json(USON, 区域多边形坐标), forbidden_roles(USON, 禁止进入的角色), safe_distance, dwell_time, is_active, created_at, updated_at
alert	告警记录表	id, household_id(→household), type(intrusion/fire/water/fall等), level(warning/critical), stream_id, description, snapshot_path, status(pending/handled), created_at, handled_at
event_log	事件日志表	id, household_id(→household), event_type, stream_id, description, metadata(JSON), created_at
sms_verification_code	短信验证码表	id, phone, code, created_at, expires_at, is_used
captcha	数字验证码表	id, question(如'3+5=7'), answer, session_key, created_at, is_used

表4-1 数据库表

5. 详细设计与实现

5.1.人脸识别模块设计

5.1.1人脸识别验证流程活动图

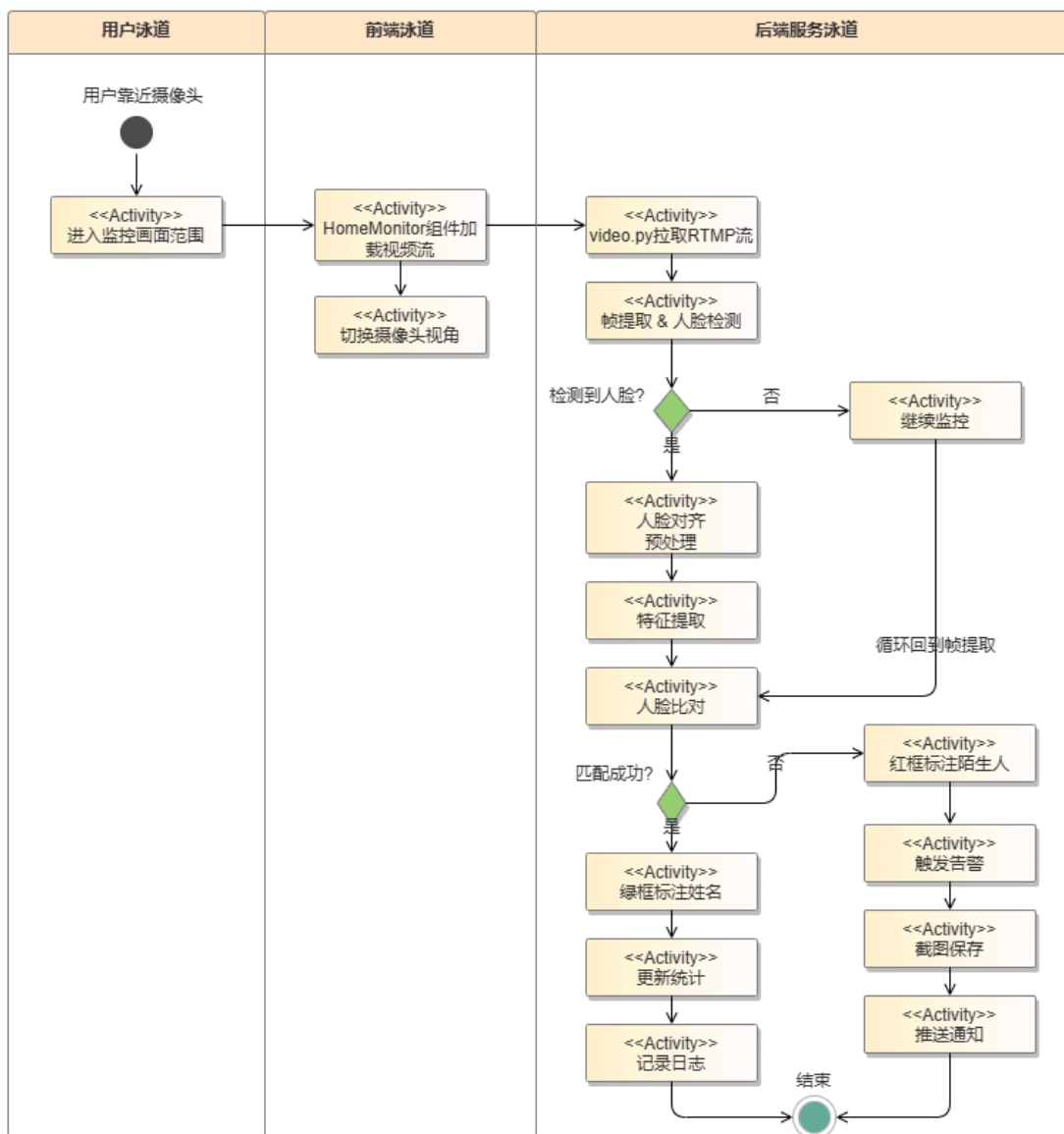


图5-1-1 人脸识别模块活动图

人脸识别模块的主体业务流程如下：video_stream 自 MediaMTX RTSP 抽帧，process_frame 链，帧数据传递给face_service进行人脸检测（dlib.get_frontal_face_detector），检测到人脸后，使用shape_predictor提取68个关键点，通过face_recognition_model计算128维人脸特征编码，与MySQL face_encoding + JSON 备份中的人脸库进行欧氏距离比对，距离小于阈值（默认0.45）判定为匹配成功，获取对应member信息。在帧图像上绘制标注框（绿框-家人/红框-陌生人），更新实时人数统计（total/family/stranger），陌生人持续出现触发FACE_UNKNOWN告警

5.1.2 人脸识别验证流程类图

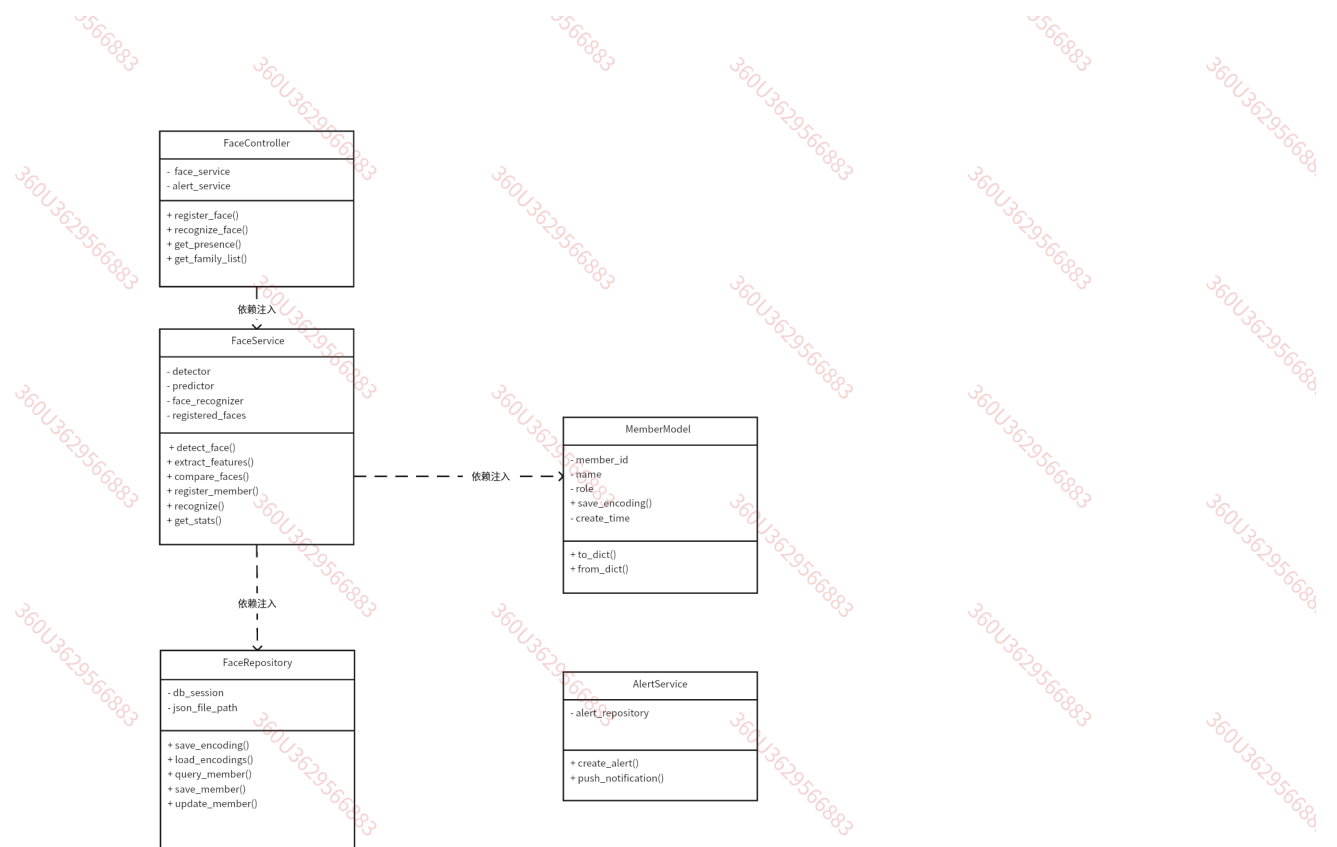


图5-1-2人脸识别模块类图

当服务器端监听到客户端发来的人脸注册接口请求时，便使用FaceController控制类进行处理。FaceController类依赖于人脸识别业务处理类FaceService，同时FaceService类依赖于人脸数据访问类FaceRepository和家庭成员实体类MemberModel，它们之间通过Django框架中的依赖注入模式进行对象传递

5.1.3人脸识别验证流程顺序图

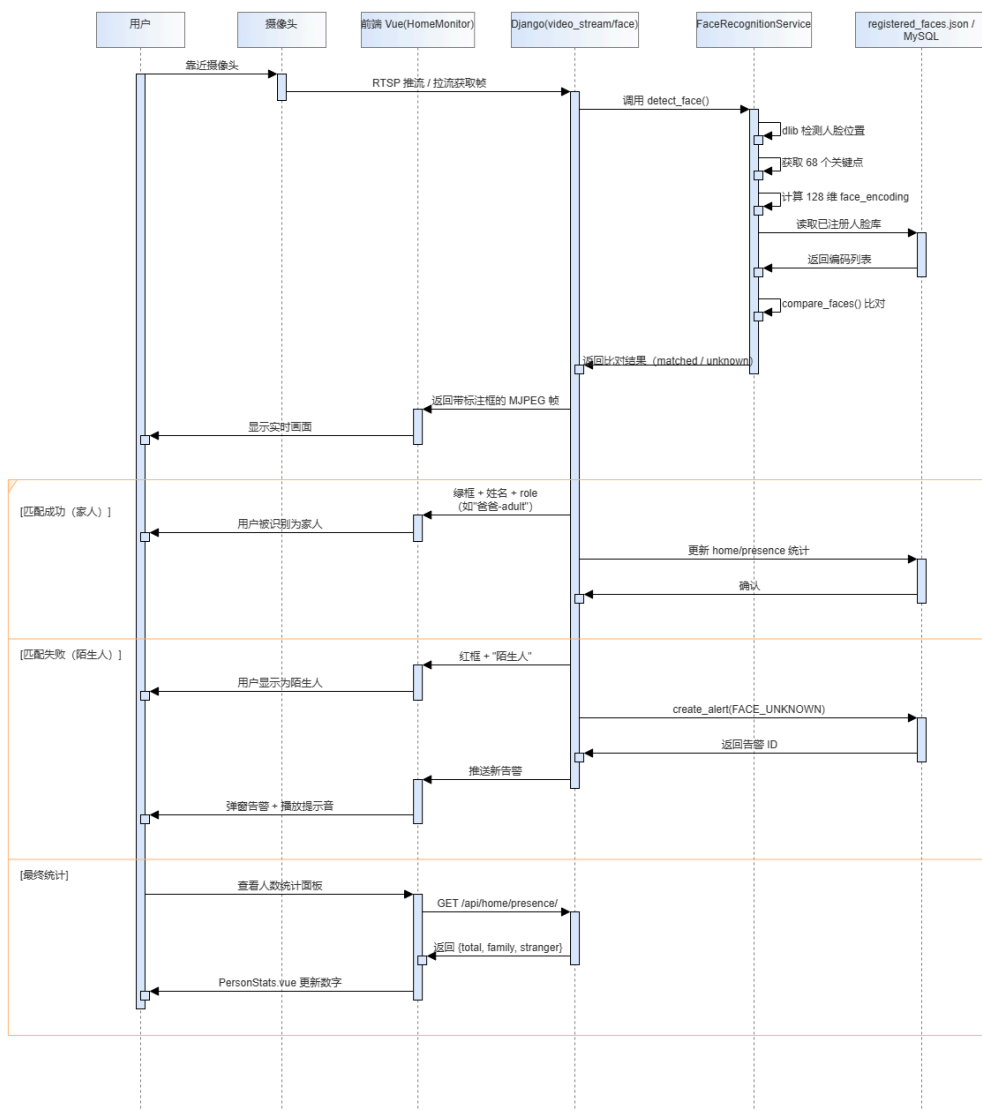
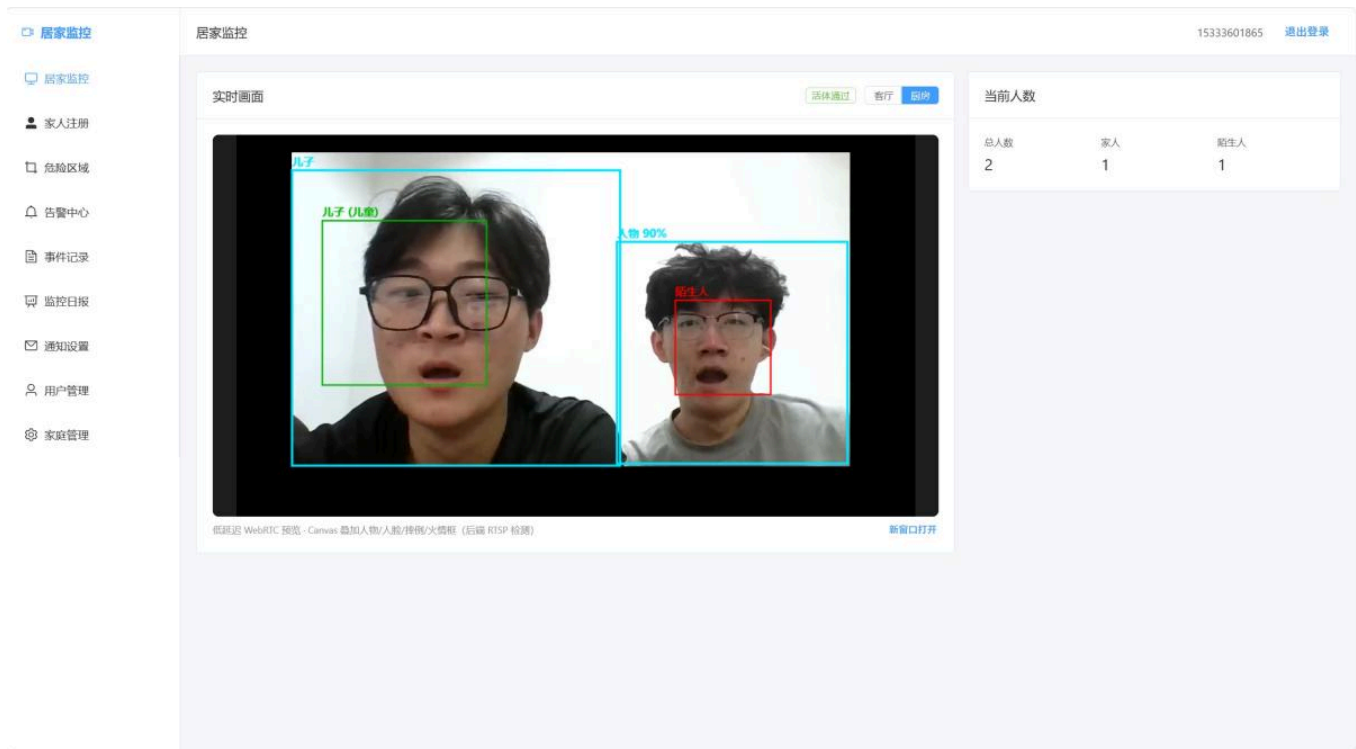


图5-1-3人脸识别模块顺序图

用户靠近摄像头，摄像头通过 RTSP 向 MediaMTX 推送实时视频流，Django video_stream 拉取视频帧后调用 FaceRecognitionService.process_frame()。FaceRecognitionService 使用 dlib.get_frontal_face_detector() 检测人脸位置，检测到人脸后通过 shape_predictor 提取 68 个关键点，再经 face_recognition_model 计算 128 维人脸特征编码。随后与 registered_faces.json 中的人脸库进行欧氏距离比对，若距离小于阈值（默认 0.45）则判定为匹配成功，获取对应 member 信息并返回绿框标注结果；若比对失败则返回红框陌生人标注。video_stream 根据识别结果更新实时人数统计（total/family/stranger），若为陌生人则持续触发 FACE_UNKNOWN 告警并调用 create_alert() 写入 MySQL 告警表。presence 供前端 Canvas 叠加；HomeMonitor 主预览为 WebRTC；MJPEG 为备用

5.1.4 人脸识别验证流程实现截图



5.2 区域管控模块设计

5.2.1 区域管控活动图

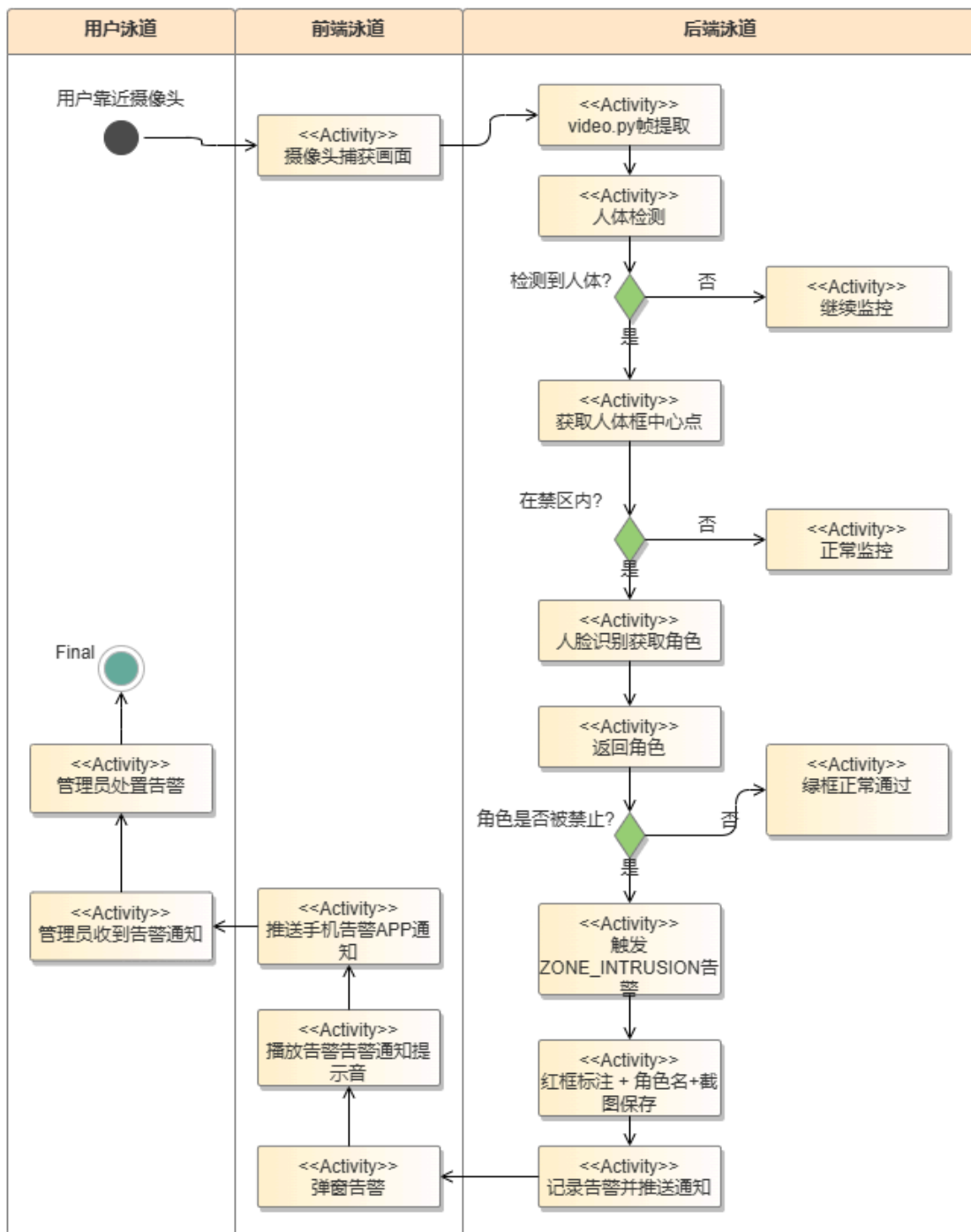


图5-2-1 区域管控活动图

区域管控模块的主体业务流程如下：管理员通过ZoneEditor.vue绘制多边形区域，设置 forbidden_roles，区域数据通过POST /api/zones保存到MySQL，实时监控时，detection_service检测人体位置（YOLOv8n 优先，HOG 降级），计算人体框中心点坐标。使用射线法判断中心点是否在区域多边形内，若在禁区内，调用face_service识别人员角色。若角色匹配forbidden_roles，触发 INTRUSION告警。告警截图保存，推送前端AlertCenter

5.2.2 区域管控类图

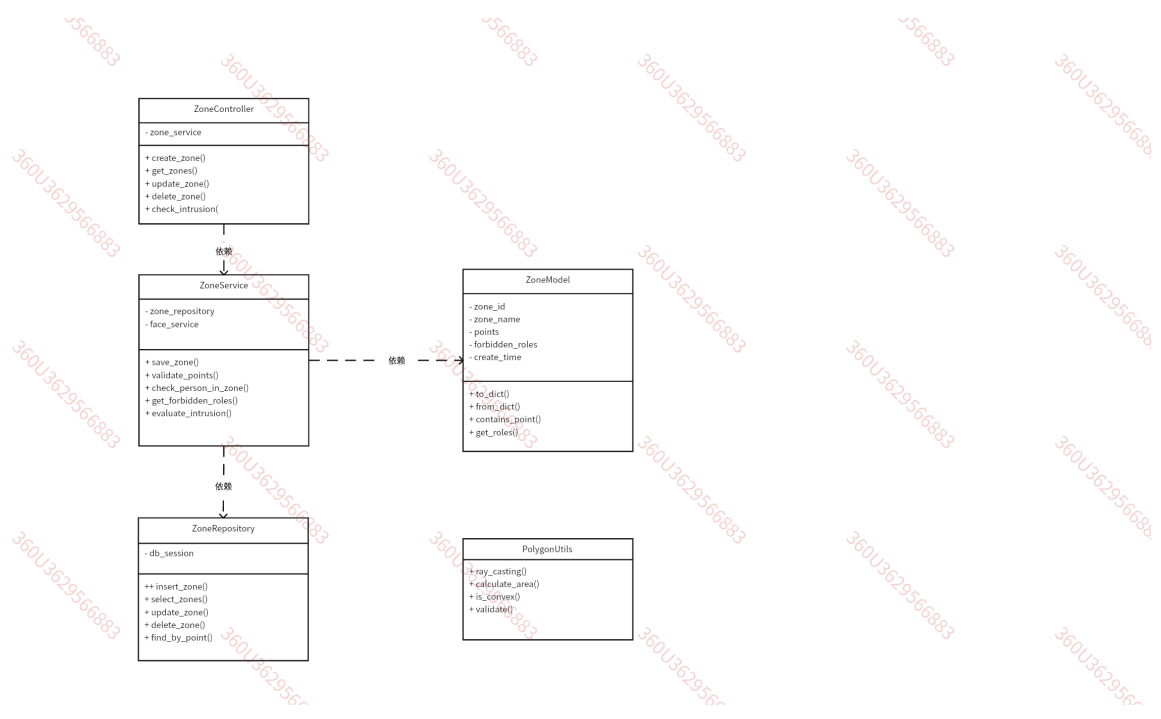


图5-2-2区域管理模块类图

当服务器端监听到客户端发来的区域管理接口请求时，便使用ZoneController控制类进行处理。ZoneController类依赖于区域业务处理类ZoneService，同时ZoneService类依赖于区域数据访问类ZoneRepository和区域实体类ZoneModel，它们之间通过Django框架中的依赖注入模式进行对象传递。

5.2.3区域管控顺序图

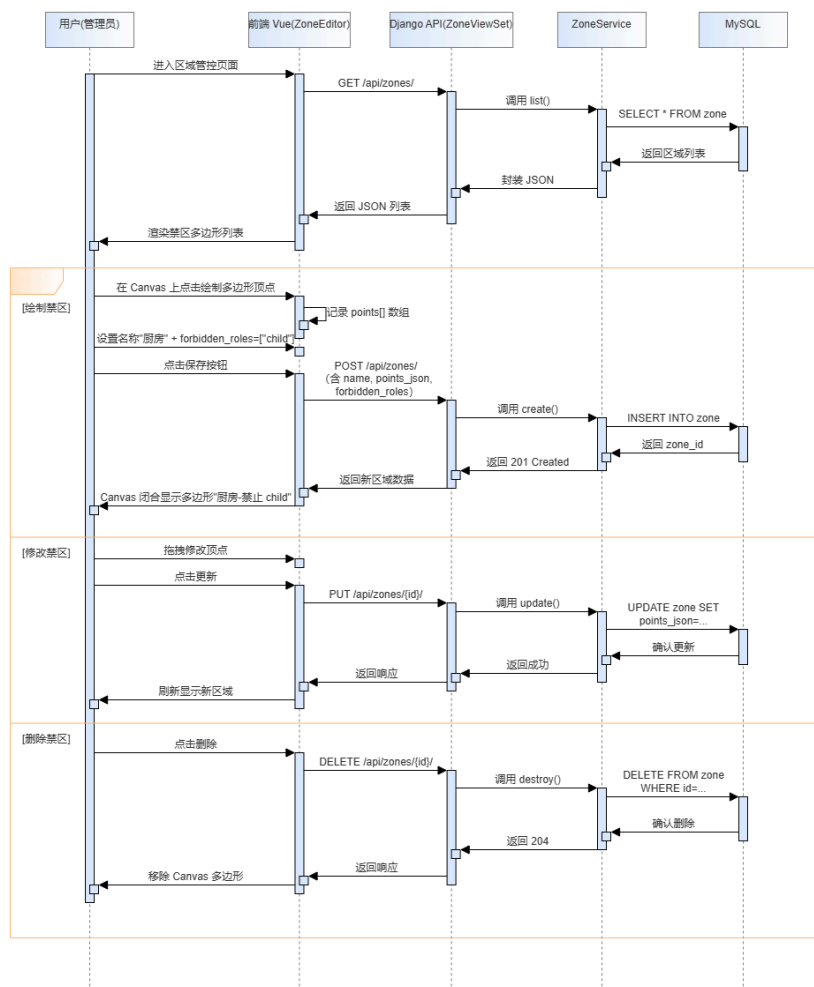
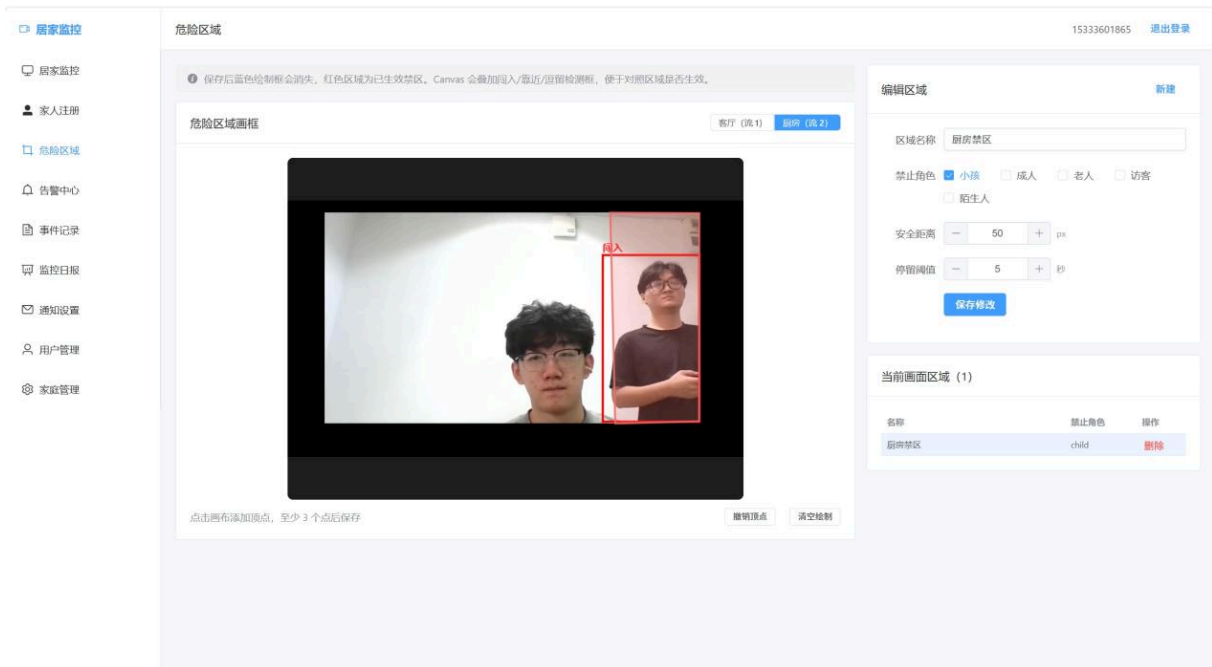
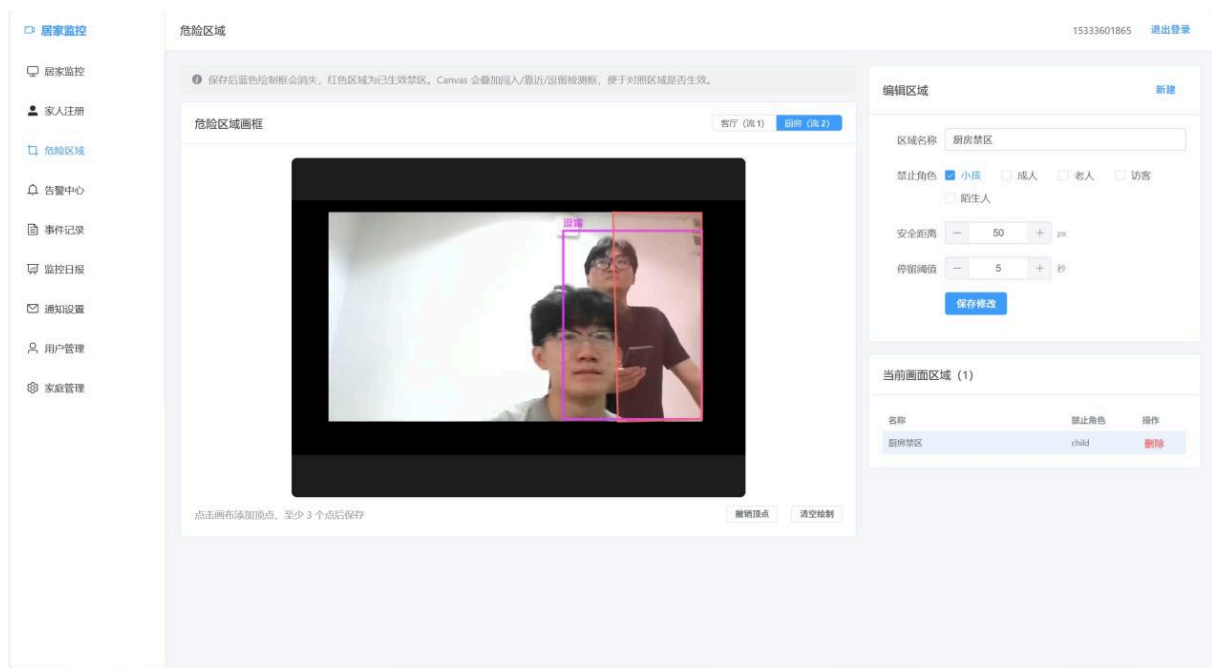


图5-2-3区域管理模块顺序图

管理员进入区域管控页面，前端 Vue(ZoneEditor) 发送 GET /api/zones/ 请求至 Django API(ZoneViewSet)。Django 调用 ZoneService.get_zones() 向 MySQL 执行 SELECT 查询，MySQL 返回区域列表后 ZoneService 封装 JSON 响应并返回前端，前端渲染禁区多边形列表。管理员在 Canvas 上点击绘制多边形顶点并记录 points[] 数组，设置禁区名称与 forbidden_roles（如「厨房」禁止「child」），点击保存后前端发送 POST /api/zones/ 至 Django，Django 调用 ZoneService.create() 向 MySQL 执行 INSERT，MySQL 返回 zone_id 后确认创建并返回 201 Created。管理员拖拽修改顶点后发送 PUT /api/zones/{id}/ 更新，Django 调用 ZoneService.update() 执行 MySQL UPDATE 并返回确认。管理员点击删除后发送 DELETE /api/zones/{id}/，Django 调用 ZoneService.destroy() 执行 MySQL DELETE 并返回 204。ZoneService 同时将 zone 配置异步推送至 DetectionService，实时比对行人位置与禁区多边形，实现闯入检测联动。

5.2.4 区域管控实现截图



5.3 异常检测模块设计

5.3.1 异常检测活动图

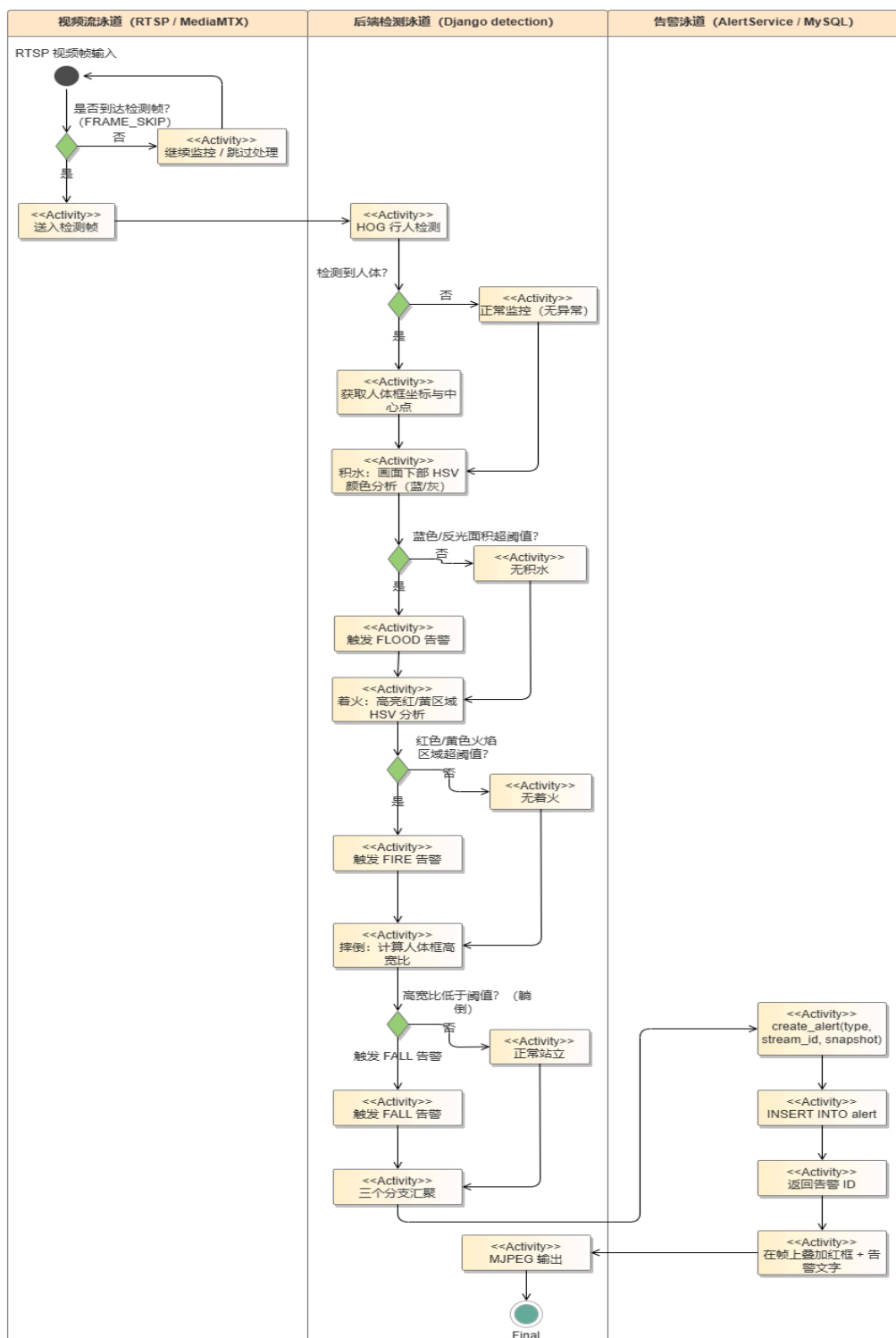


图5-3-1异常检测模块活动图

video_stream 从 RTSP 视频流中提取帧数据，按照 FRAME_SKIP 跳帧策略筛选后送入 DetectionService。首先调用 YOLOv8n 优先，HOG 降级进行行人检测，若未检测到人体则返回继续监控；检测到人体后获取人体框坐标与中心点，依次执行2种异常检测分支：着火检测通过高亮红/黄火焰颜色区域超阈值判定 FIRE；摔倒检测通过人体框高宽比低于阈值判定 FALL。任一异常触发后，调用 create_alert() 将告警类型、stream_id 与截图路径写入 MySQL 告警表，同时在帧图像上叠加红框标注与告警文字，最终经 create_alert + WebRTC/Canvas输出至前端浏览器实时展示。

5.3.2异常检测类图

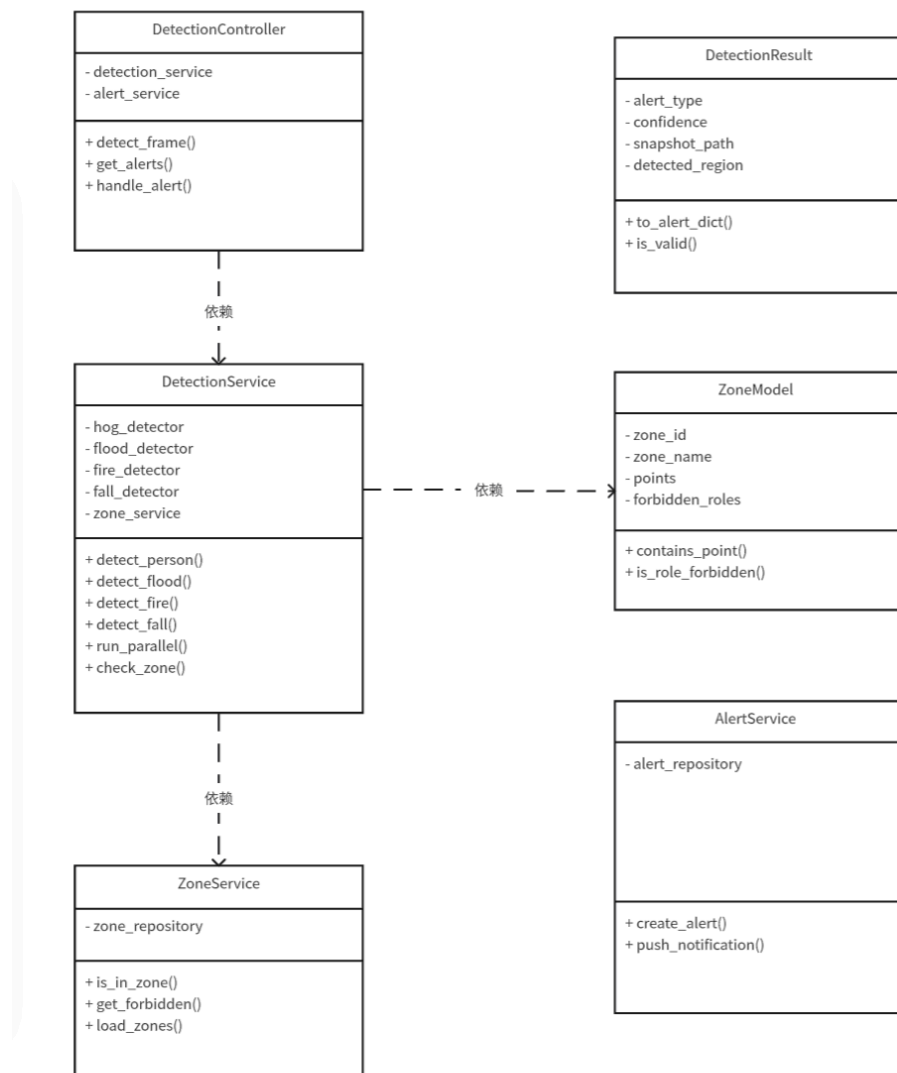


图5-3-2异常检测模块类图

当服务器端监听到客户端发来的视频帧检测请求时，便使用DetectionController控制类进行处理。DetectionController类依赖于异常检测业务处理类DetectionService，同时DetectionService类依赖于区域服务类ZoneService和区域实体类ZoneModel，它们之间通过Django框架中的依赖注入模式进行对象传递。

5.3.3异常检测顺序图

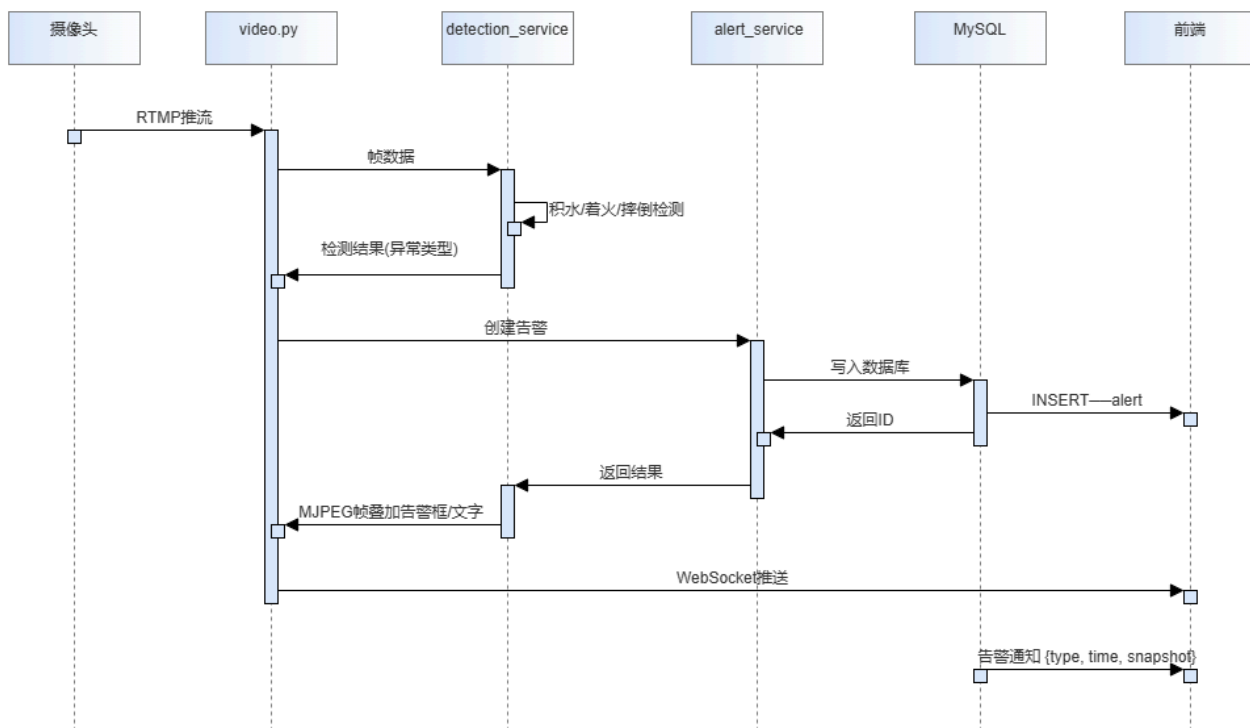
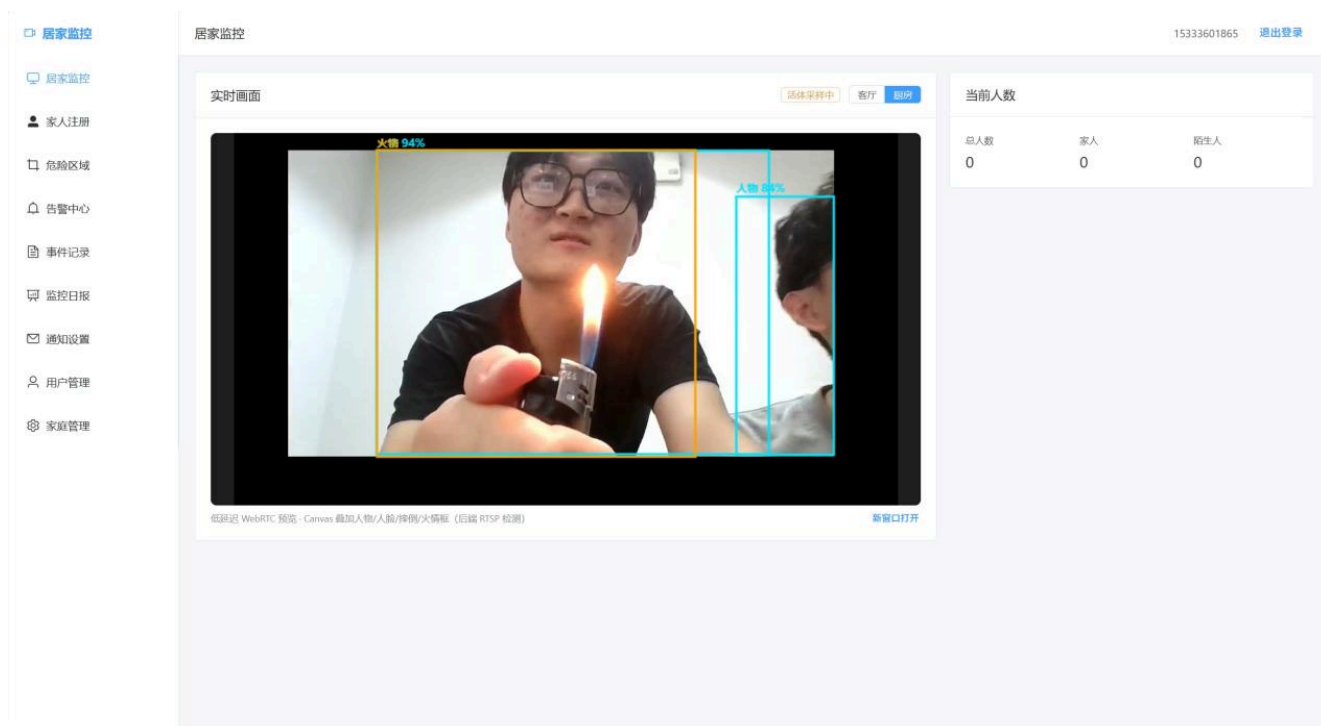
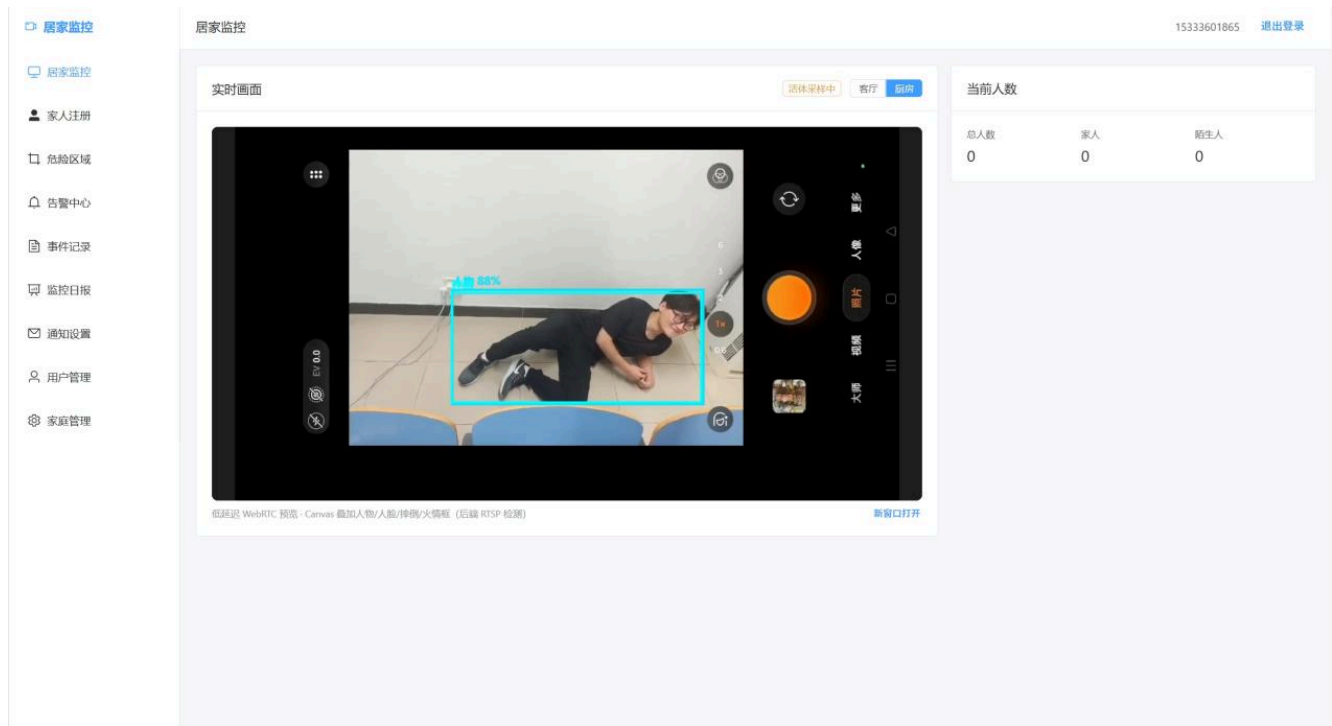


图5-3-3异常检测告警顺序图

用户出现在摄像头画面中，摄像头通过 RTSP 向 MediaMTX 推流，Django video_stream 拉取视频帧后调用 DetectionService.process_frame()。DetectionService 首先使用 YOLO 优先检测人体并获取人体框坐标与中心点；随后并行执行两种异常检测算法：着火检测分析高亮红/黄区域是否超阈值，摔倒检测计算人体框高宽比是否低于躺倒阈值。若任一异常条件满足，DetectionService 调用 create_alert() 向 MySQL 写入告警记录，同时返回告警信息至 video_stream，在视频帧上叠加红框与告警文字后，最终经过create_alert + WebRTC/Canvas输出至前端浏览器实时展示

5.3.4异常检测实现截图





5.4 视频监控模块设计

5.4.1 视频监控活动图

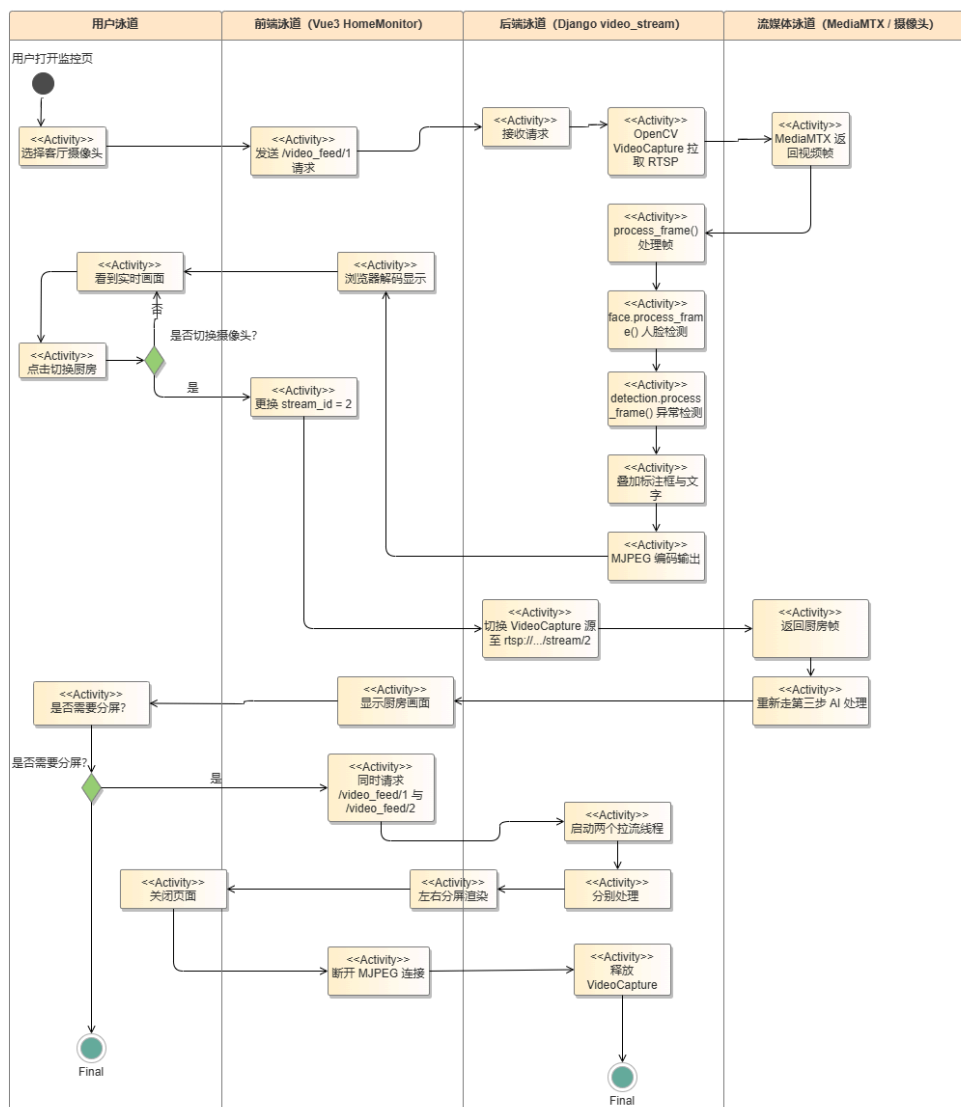


图5-4-1视频监控模块活动图

用户打开 HomeMonitor 监控页并选择客厅或厨房摄像头。

前端以 WebRTC iframe 加载 MediaMTX 发布的低延迟画面（如 `http://{服务器IP}:8889/stream/{id}/`），用于显示原始实时预览。

同时，前端约每 200ms 轮询 `GET /api/video/presence/?stream_id={id}`，获取人脸框、行人/告警框及人数统计；`FaceOverlay` 在 Canvas 上叠加 AI 标注，`PersonStats` 展示 total / family / stranger。

后端侧，摄像头推流经 RTMP 进入 MediaMTX；`CameraWorker`（或等价拉流服务）从 MediaMTX 拉取 RTSP 帧，在独立线程中执行处理链：先 `face.process_frame()`（检测、128 维编码、家属比对），再 `detection.process_frame()`（YOLOv8n 优先、HOG 降级，区域闯入/靠近/停留及着火、摔倒等异常判断），再发布 `overlay/presence` 数据供前端查询。告警经 `create_alert()` 落库。

`/video_feed/{id}` 的 MJPEG 仍保留，用于调试或备用（帧上可烧录标注），不是 HomeMonitor 的主预览路径。

用户切换摄像头时，前端更换 WebRTC 的 `stream_id` 并改 `presence` 轮询参数；开启分屏时并行加载多路 WebRTC 与多路 `presence`。关闭页面后停止轮询并释放相关资源；停推流约数秒后 `presence`

清空，页面可提示「无推流信号」。

5.4.2视频监控类图

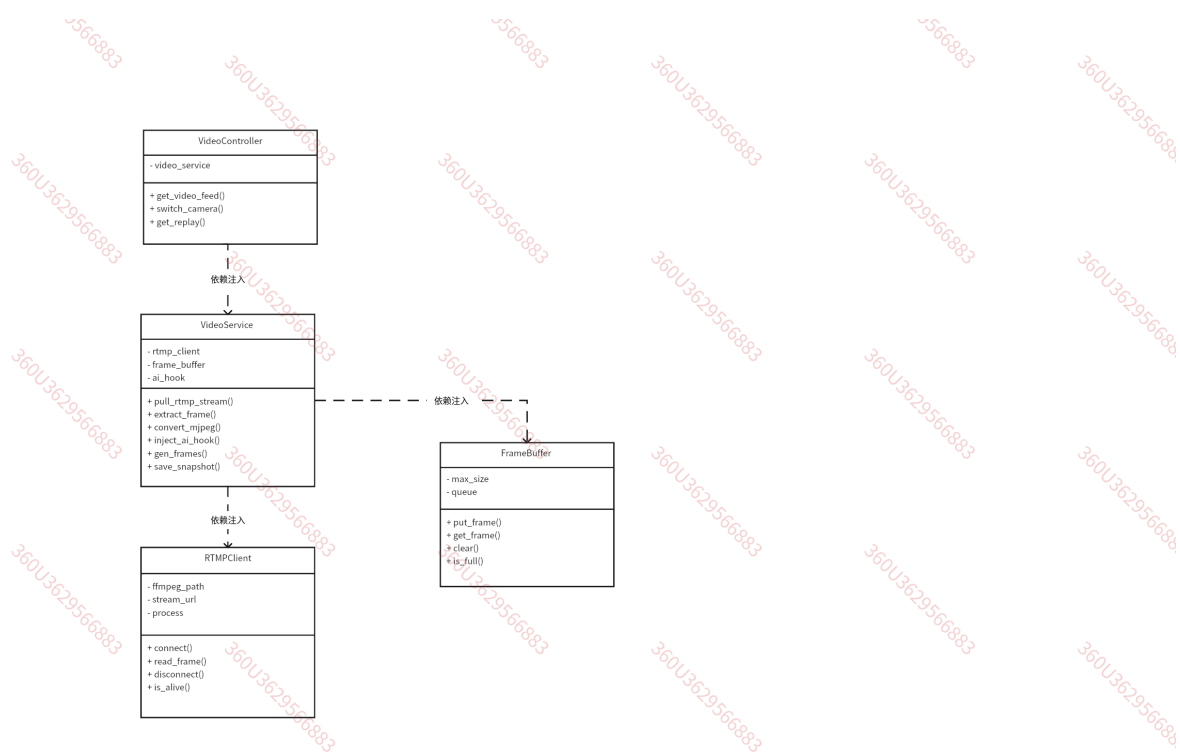


图5-4-2视频监控模块类图

当服务器端监听到客户端发来的视频流请求时，便使用VideoController控制类进行处理。VideoController类依赖于视频业务处理类VideoService，同时VideoService类依赖于RTMP客户端类RTMPCClient和帧缓冲类FrameBuffer，它们之间通过Django框架中的依赖注入模式进行对象传递。

5.4.3视频监控顺序图

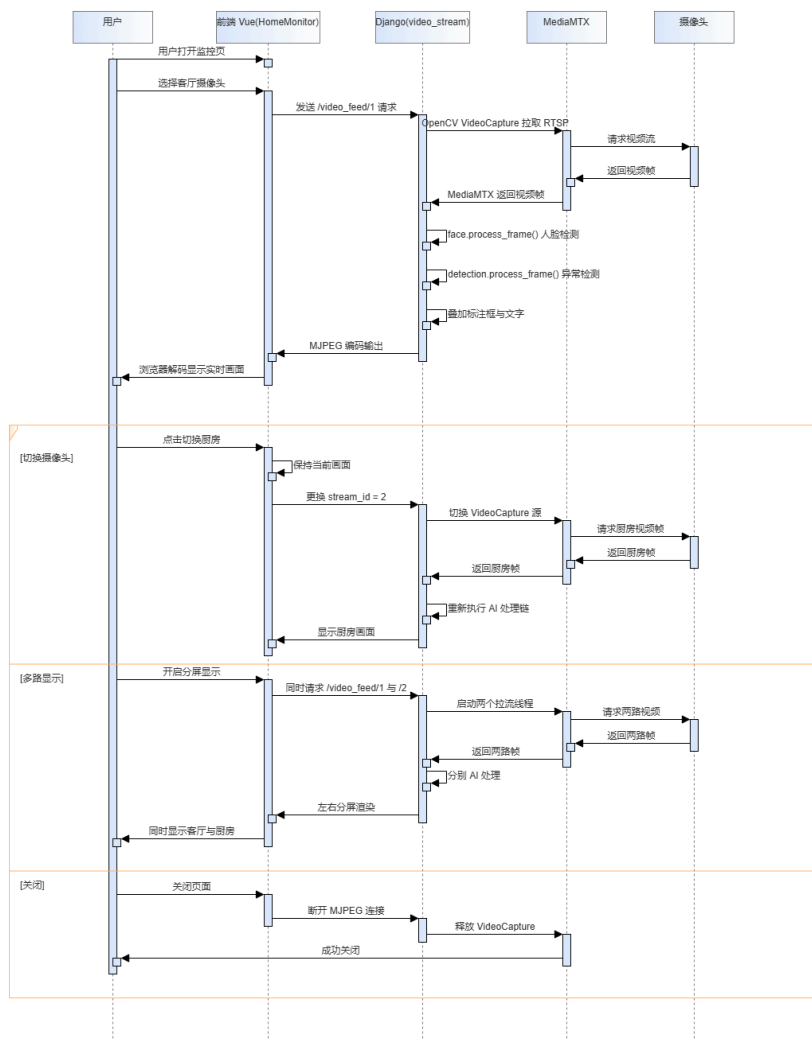
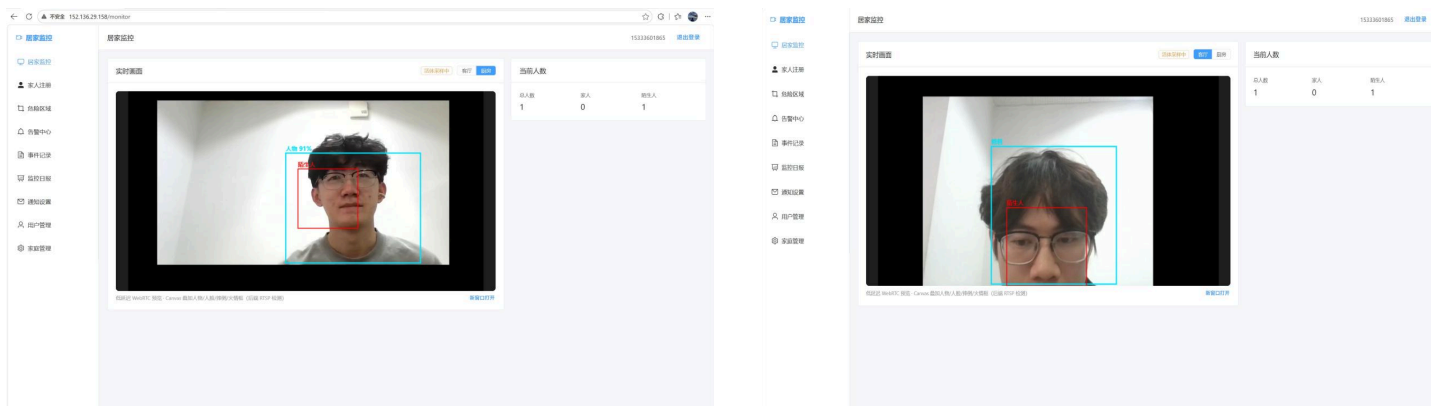


图5-4-3视频监控模块顺序图

用户打开监控页并选择客厅摄像头，前端 Vue(HomeMonitor)WebRTC iframe + 轮询 presence。Django 通过 OpenCV VideoCapture 从 MediaMTX 的 RTSP 地址拉取视频帧，依次调用 face.process_frame() 进行人脸检测与 128 维编码比对识别家人或陌生人，调用 detection.process_frame() 进行行人检测与异常判断，再调用 detection.draw_overlays() 叠加区域多边形与告警框。以 WebRTC iframe 加载 MediaMTX 发布的低延迟画面推送至前端，浏览器实时解码显示。用户切换厨房摄像头时，前端更换 stream_id 为 2，后端切换 VideoCapture 源重新拉流并执行 AI 处理链；用户开启分屏时，前端同时请求两路视频流，后端启动两个拉流线程分别处理并返回。

5.4.4视频监控实现截图



5.5 告警中心模块设计

5.5.1告警中心活动图

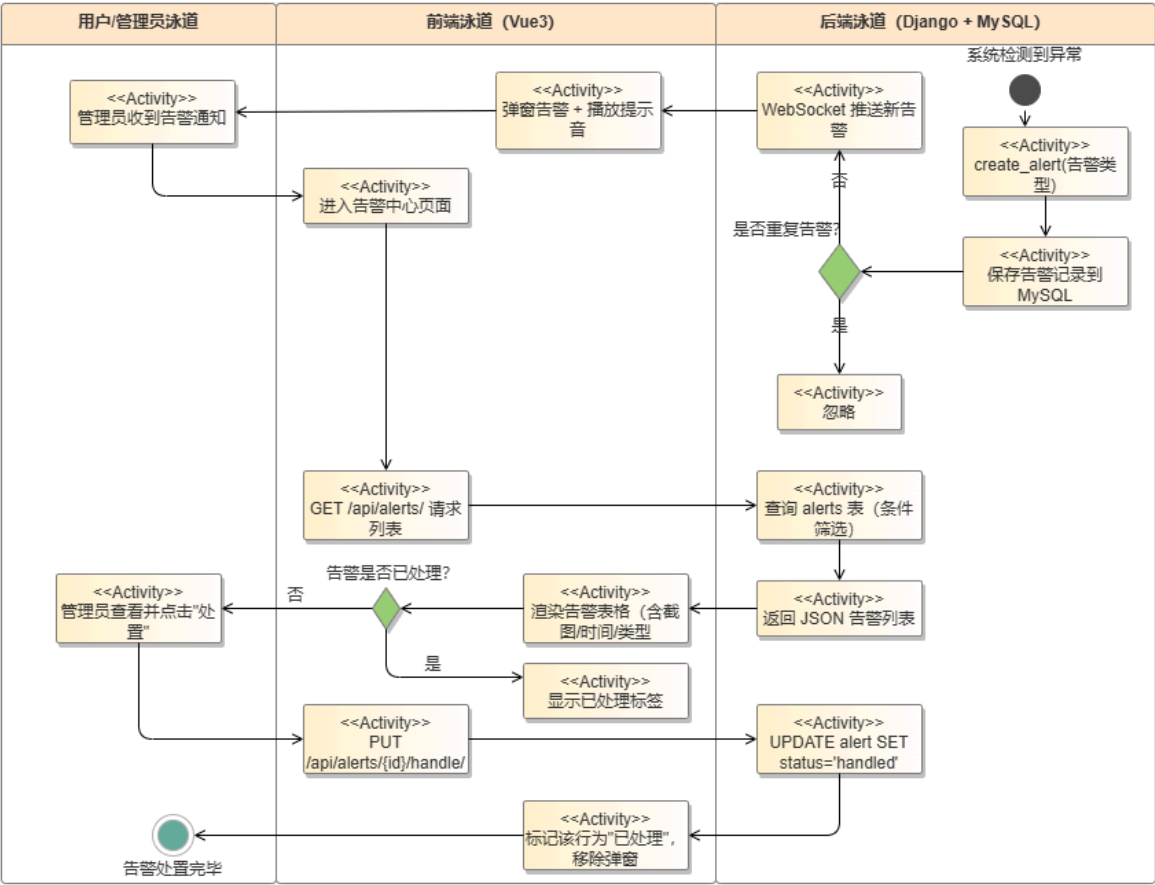


图5-5-1告警中心模块活动图

管理员进入 AlertCenter 告警中心，前端发送 GET /api/alerts/ 请求获取告警列表，Django 后端调用 get_alert_list() 向 MySQL 执行条件查询，封装 JSON 响应后返回前端渲染告警表格。管理员可按类型（陌生人/闯入/着火/摔倒）筛选告警，前端发送带筛选参数请求重新查询数据库。当 AI 模块触发新告警时，前端约 3s 轮询刷新列表，前端弹窗并播放提示音通过钉钉通知管理员。管理员点击"处置"按钮后，前端发送 PUT /api/alerts/{id}/handle/，后端调用 handle_alert() 更新 MySQL 告警表状态为 handled，确认更新后前端移除弹窗并标记已处理。

5.5.2告警中心类图

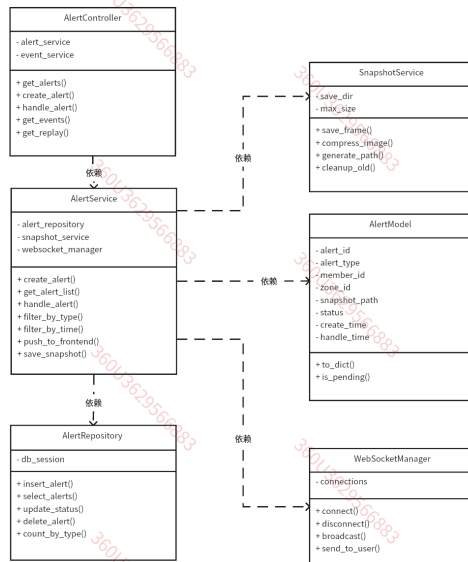


图5-5-2告警中心模块类图

当服务器端监听到客户端发来的告警查询请求时，便使用AlertController控制类进行处理。AlertController类依赖于告警业务处理类AlertService，同时AlertService类依赖于告警数据访问类AlertRepository、截图服务类SnapshotService和WebSocket管理类WebSocketManager，它们之间通过Django框架中的依赖注入模式进行对象传递。

5.5.3告警中心顺序图

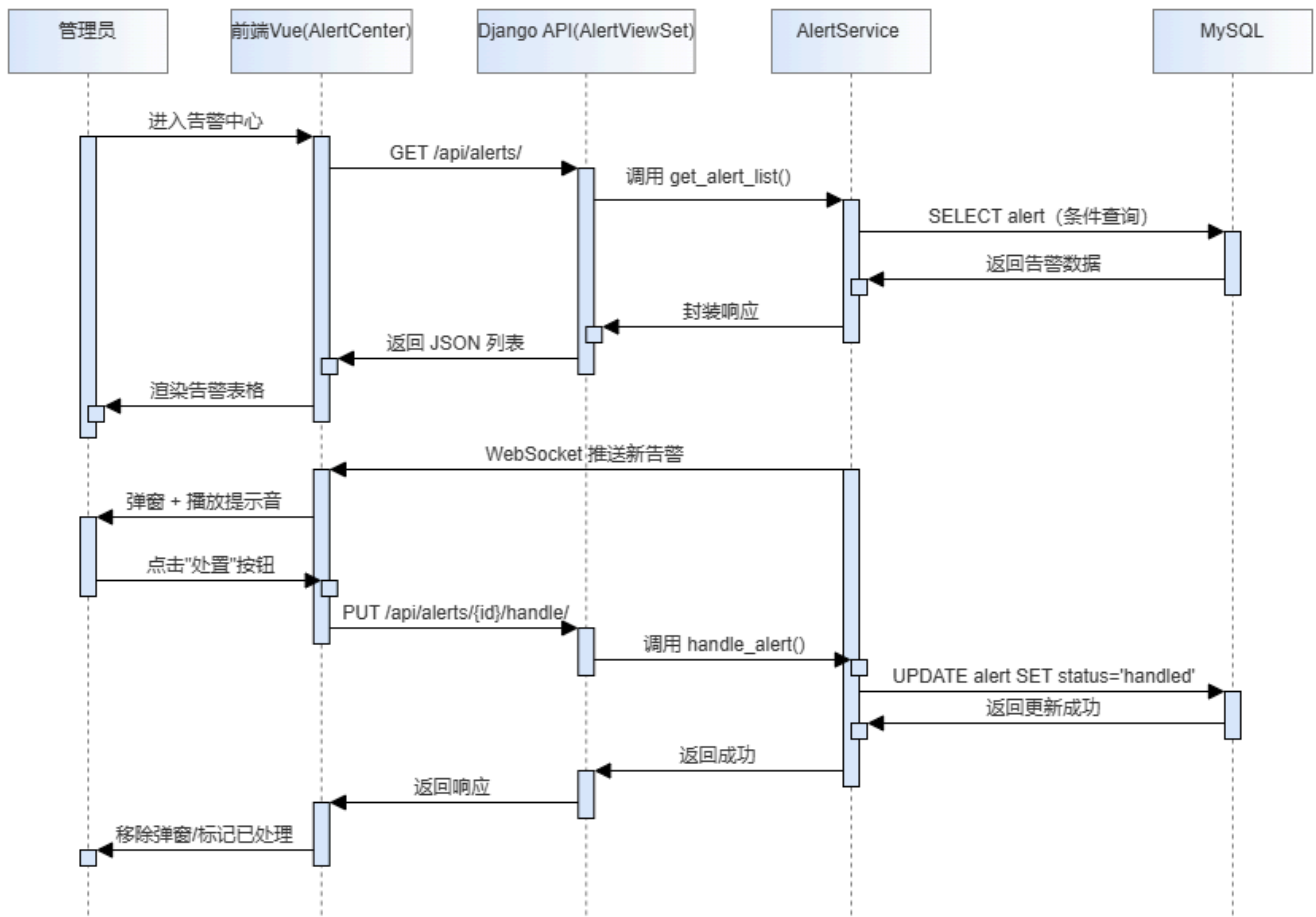
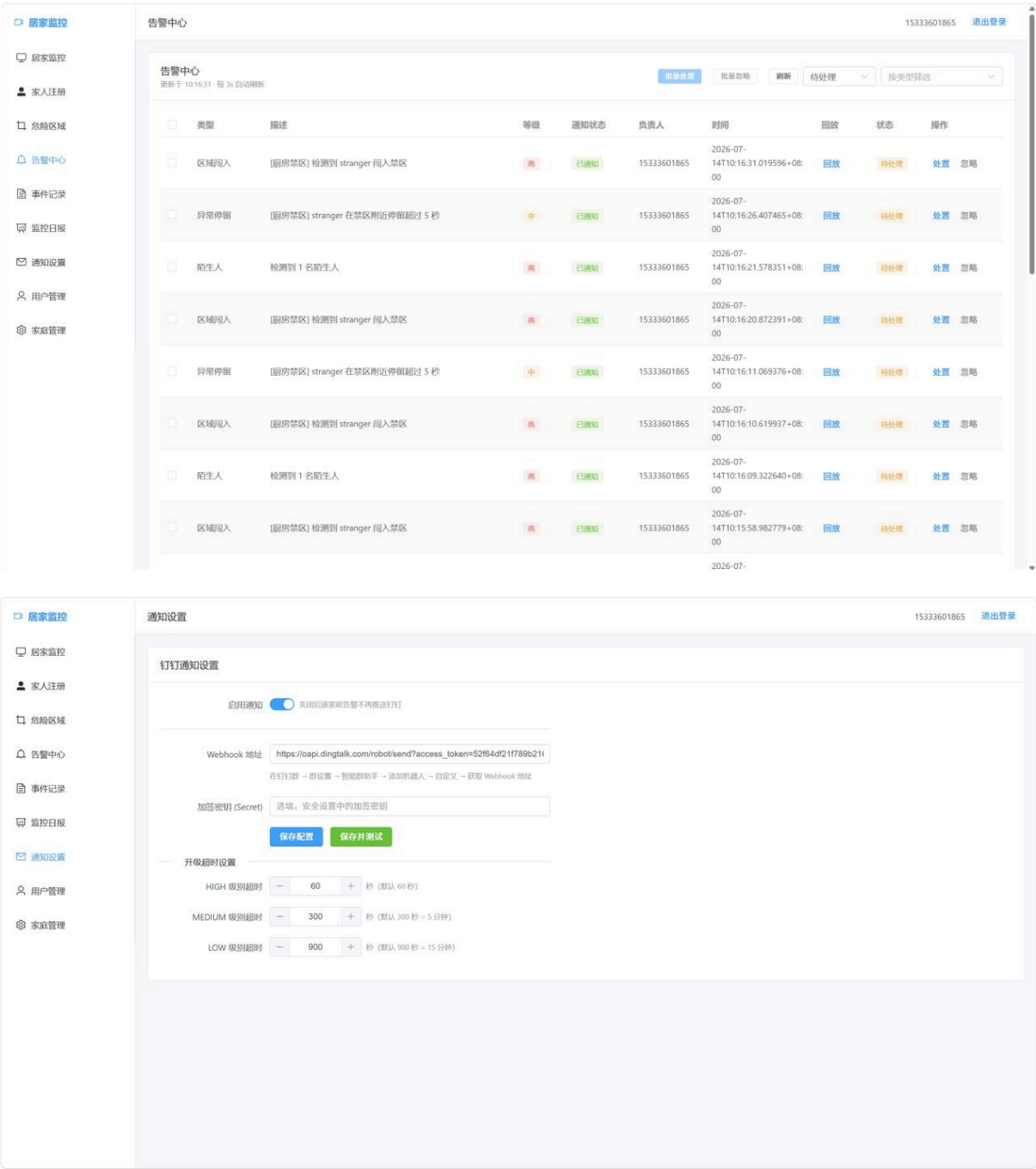
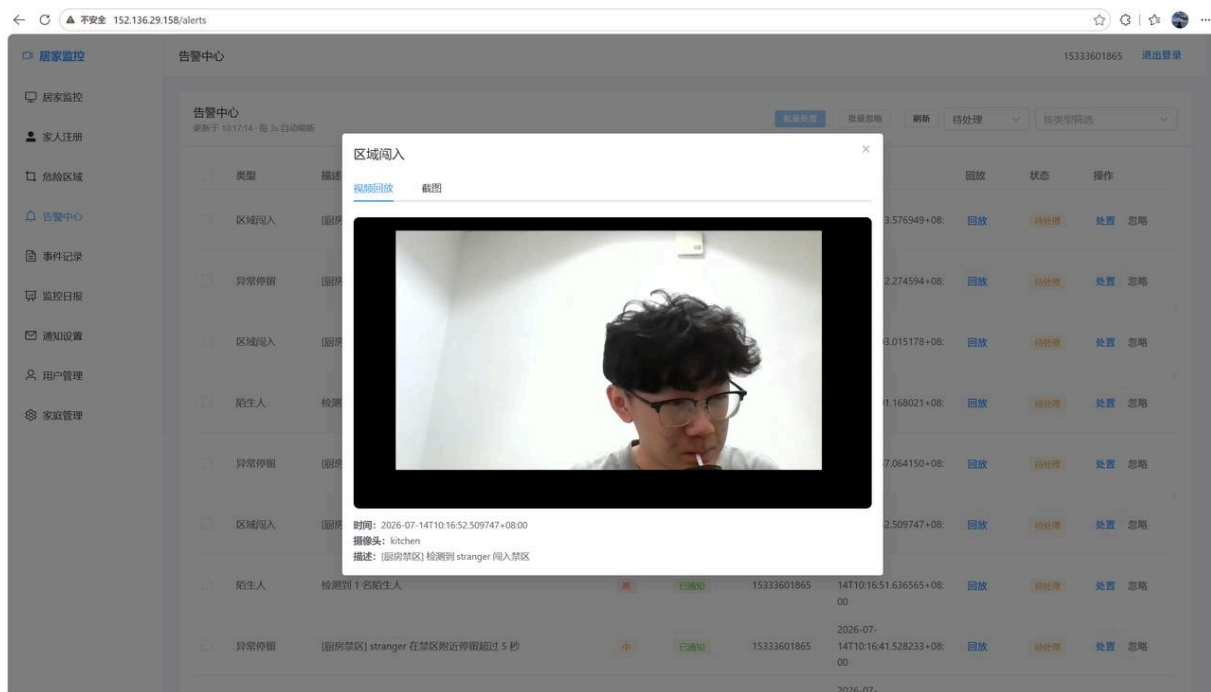
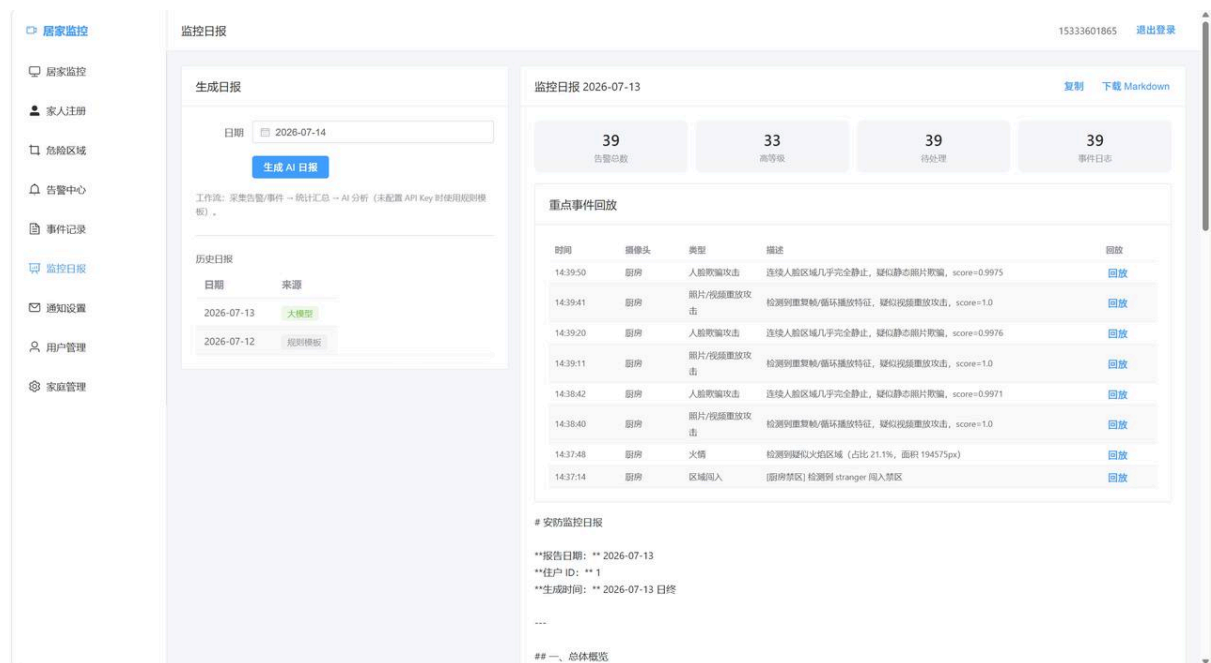


图5-5-3告警中心模块类图

管理员进入告警中心，前端 Vue(AlertCenter) 发送 GET /api/alerts/ 至 Django API(AlertViewSet)。Django 调用 AlertService.get_alert_list() 向 MySQL 执行 SELECT 条件查询，MySQL 返回告警数据后 AlertService 封装响应并返回 JSON 列表至前端，前端渲染告警表格。当 AI 模块触发新告警时，前端约 3s 轮询 GET /api/alerts/ 推送至前端，前端弹窗并播放提示音。管理员点击"处置"按钮后，前端发送 PUT /api/alerts/{id}/handle/，Django 调用 AlertService.handle_alert()，MySQL 执行 UPDATE 将 status 设为 handled，返回更新成功后前端移除弹窗并标记已处理。

5.5.4告警中心实现截图





5.6 用户管理模块设计

5.6.1 用户管理活动图

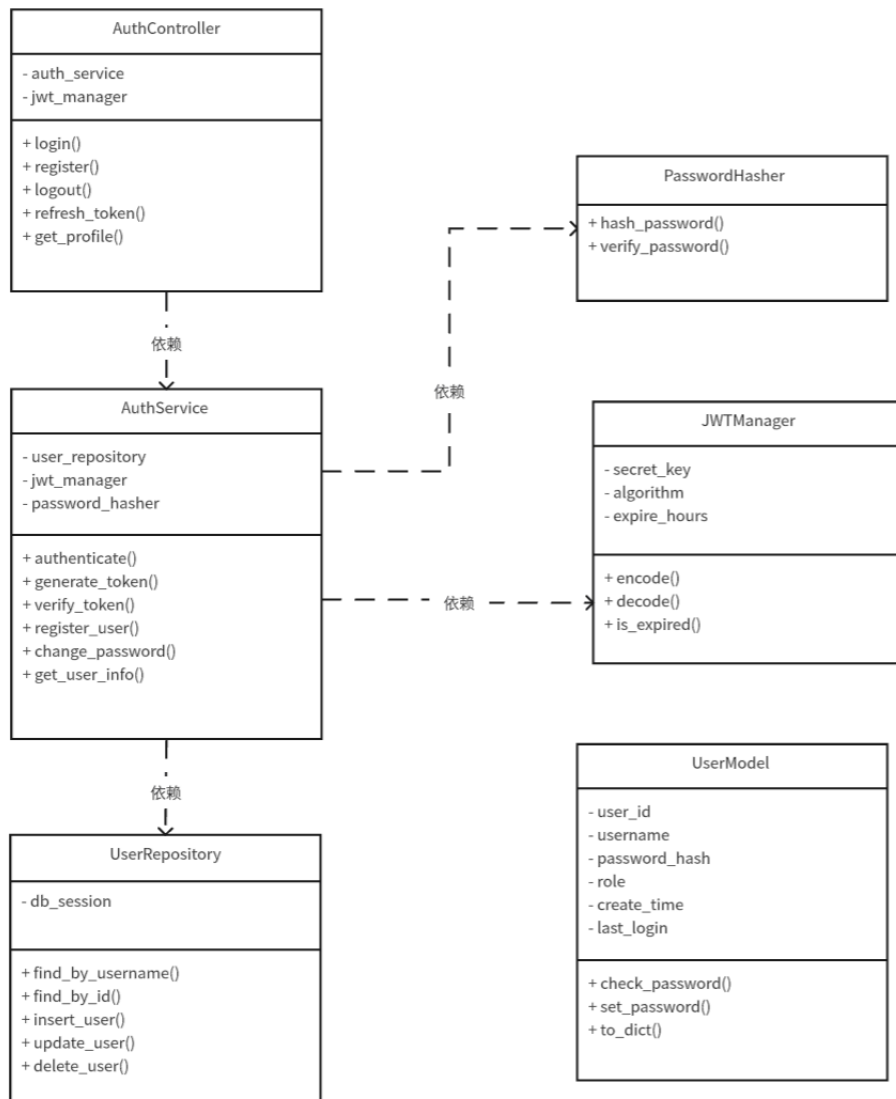


图5-6-2用户管理模块类图

当服务器端监听到客户端发来的登录请求时，便使用AuthController控制类进行处理。AuthController类依赖于认证业务处理类AuthService，同时AuthService类依赖于用户数据访问类UserRepository、JWT管理类JWTManager和密码加密类PasswordHasher，它们之间通过Django框架中的依赖注入模式进行对象传递。

5.6.3用户管理顺序图

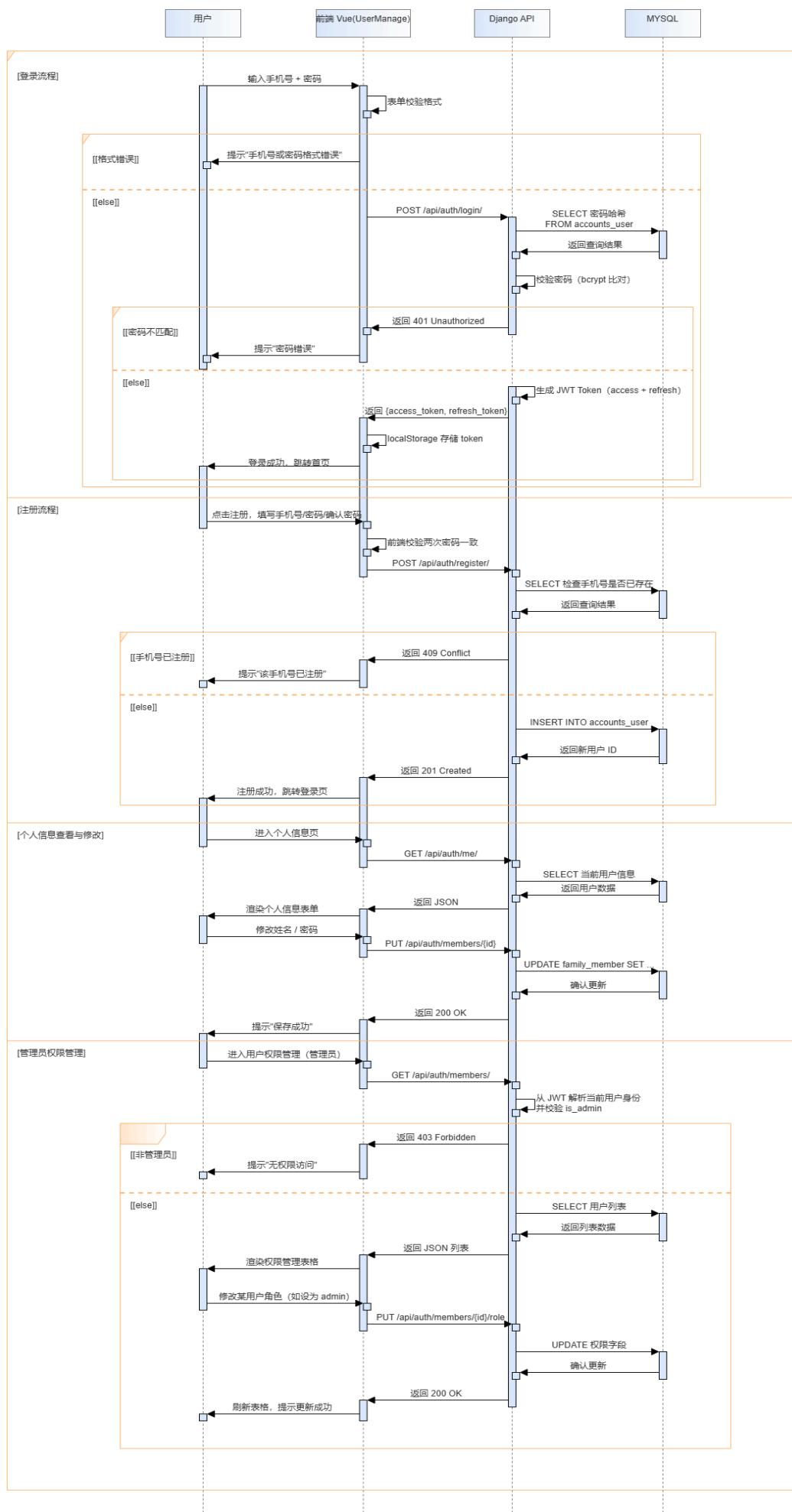


图5-6-3用户管理模块顺序图

用户输入手机号与密码，前端 Vue(UserManage) 进行格式校验后发送 POST /api/auth/login/ 至 Django API。Django 查询 MySQL 获取密码哈希并执行校验，验证通过后生成 JWT Token 返回前端，前端存储 token 并登录成功。用户进入个人信息页时，前端发送 GET /api/auth/me/，Django 查询 MySQL 返回当前用户信息，前端渲染表单。用户修改信息后发送 PUT /api/auth/members/{id}，Django 更新 MySQL 中 family_member 表并返回成功。管理员进入权限管理时，前端发送 GET /api/auth/members/，Django 校验管理员身份后查询 MySQL 用户列表并返回，前端渲染表格；管理员修改角色后发送 PUT 更新请求，Django 更新 MySQL 权限字段并返回确认。

5.6.4用户管理实现截图

居家监控

居家监控

家人注册

危险区域

告警中心

事件记录

监控日报

通知设置

用户管理

家庭管理

家庭管理

已切换到「三哥家」

15333601865 退出登录

我的家庭

创建新家庭 申请加入

ID	家庭名称	角色	状态	操作
1	三哥家	管理员	当前	进入 成员管理 审核 删除

三哥家 — 家庭成员 2人

姓名	身份	角色	人脸	操作
牛雨昊	爸爸	成人	已录入	编辑 删除
刘帅华	儿子	小孩	已录入	编辑 删除

居家监控

居家监控

家人注册

危险区域

告警中心

事件记录

监控日报

通知设置

用户管理

家庭管理

个人信息

手机号 15333601865

我的钉钉 (首次通知 @ 我)

钉钉 UserID 牛雨昊

钉钉手机号 15333601865

上级钉钉 (超时升级 @ 上级)

上级 UserID 刘帅华

上级手机号 13783222587

告警超时未处理时，系统会向群推送升级消息并 @ 上级。
上级不必是本系统用户，填写对方的钉钉 UserID 即可。

保存 更换手机号

6.项目结构

6.1 核心API清单

路径	方法	说明
/api/auth/login/	POST	JWT 登录
/api/auth/register/	POST	手机号注册
/api/auth/members/	CRUD	家庭成员管理
/api/auth/me/	GET	当前用户信息
/api/households/	CRUD	家庭管理
/api/households/join/	POST	申请加入家庭
/api/cameras/	CRUD	摄像头管理
/api/home/presence/	GET	当前人数统计
/api/zones/	CRUD	危险区域
/api/alerts/	GET/POST	告警列表 / 创建
/api/alerts/{id}/handle/	PUT	处置告警
/api/events/	GET/POST	事件日志
/api/face/register/	POST	人脸注册（多帧活体校验）
/api/face/liveness/	POST	多帧活体检测
/api/face/analyze/	POST	单帧人脸分析（调试用）
/api/detection/analyze/	POST	单帧异常检测（调试用）
/api/detection/status/	GET	检测服务状态
/api/detection/audio/status/	GET	音频检测状态
/api/video/presence/	GET	实时 overlay + 人数统计
/video_feed/{id}	GET	MJPEG 备用流（含后端烧录标注）

/api/video/status	GET	视频 worker 状态
/api/notifications/config/	GET/PUT	钉钉通知配置
/api/video/streams/{id}/source	GET	推流/拉流地址信息
/api/reports/daily/	GET/POST	AI 监控日报
/api/snapshots/{filename}/	GET	事件回放截图
/api/docs/	GET	Swagger 文档

6.2模块职责

模块	职责	负责人
accounts	手机号注册/登录、JWT、家庭成员 CRUD	刘帅华
households	家庭创建/加入/审核、摄像头管理、多家庭隔离	刘帅华
home	人数统计 API（读取 face 服务快照）	王梓铭 / 刘帅华
zones	危险区域 CRUD（多边形 + forbidden_roles）	刘帅华
alerts	告警管理、create_alert 服务层	刘帅华
events	事件日志	刘帅华
video_stream	RTSP 拉流、AI 处理链、overlay API、MJPEG 备用输出	苏哲勋
face	人脸注册/识别/活体反欺骗/人数统计/陌生人告警	王梓铭
detection	禁区闯入、着火/跌倒、音频异常、音视频联动	李东礼
notifications	钉钉 Webhook 推送、告警升级链	刘帅华
reports	AI 监控日报生成	刘帅华 / 牛雨昊

6.3仓库目录

home-camera-monitor/
├── frontend/ # Vue3 前端 (:5173)
│ └── src/
│ └── views/ # 页面组件

- | |—— components/ # 公共组件 (PersonStats 等)
- | |—— api/ # Axios 接口封装
- | |—— constants/ # streams.js 流 ID 映射
- | |—— router/ # Vue Router
- |—— backend/ # Django 后端 (:8000)
 - | |—— config/ # settings / urls / wsgi
 - | |—— apps/ # 业务 + AI Django Apps (见 7.2)
 - | |—— dat/ # dlib 模型说明
 - | |—— environment.yml # conda 环境 (Python 3.10 + dlib)
 - | |—— requirements.txt
- |—— nginx/ # Nginx / MediaMTX 部署说明
- |—— deploy/ # 部署脚本 (见 deploy/README.md)
- |—— .github/workflows/ # GitHub Actions CI
- |—— Jenkinsfile # Jenkins Pipeline (CD)
- |—— openspec/ # OpenSpec 规格驱动开发
- |—— docs/ # 项目文档 (见 docs/README.md)

6.4 后端 Django Apps

backend/apps/

- |—— accounts/ # 用户认证 (手机号 + JWT)、家庭成员 CRUD
 - | |—— models.py # User, FamilyMember, SmsVerificationCode, Captcha
 - | |—— management/commands/seed_demo_data.py
- |—— households/ # 家庭管理、成员关系、加入申请、摄像头
 - | |—— models.py # Household, HouseholdMembership, JoinApplication, Camera
 - | |—— middleware.py # 活跃家庭上下文
 - | |—— filters.py # resolve_active_household_id()
- |—— home/ # 居家监控
 - | |—— views.py # GET /api/home/presence/
- |—— zones/ # 危险区域 CRUD
- |—— alerts/ # 告警列表 / 创建 / 处置
 - | |—— services.py # create_alert() 供 AI 模块调用

- └── events/ # 事件日志
- └── video_stream/ # 视频拉流 + AI 处理 + MJPEG 输出 (B-苏哲勋)
 - └── services.py # CameraWorker、process_frame()、overlay 快照
 - └── views.py # /video_feed/、/api/video/status、/api/video/presence/
 - └── clips.py # 告警短视频录制
- └── face/ # 人脸识别 + 活体反欺骗 (C-王梓铭)
 - └── services.py # FaceRecognitionService
 - └── liveness.py # 被动活体检测
 - └── views.py # /api/face/register|analyze|liveness/
 - └── tests/
- └── detection/ # 危险区域 + 异常 + 音频检测 (D-李东礼)
 - └── services.py # DetectionService (YOLO/HOG + 禁区/异常)
 - └── audio_service.py # PANNs 音频分类
 - └── audio_capture.py # FFmpeg 音频采集
 - └── av_correlation.py # 音视频联动 EMERGENCY
 - └── tests/
- └── notifications/ # 钉钉通知 + 告警升级链 (E-刘帅华)
 - └── views.py # /api/notifications/config/
 - └── signals.py # 告警推送
 - └── checker.py # 超时逐级上报
- └── reports/ # AI 监控日报 (A/E)
- └── services.py # 日报生成 (模板 / 可选 LLM)

7.团队分工

代号	Github昵称	成员	角色
A	niu-spec	牛雨昊	组长 + 前端 + Git/部署
B	sober	苏哲勋	流媒体 + 视频拉流
C	awsladaw	王梓铭	AI 人脸识别 + 人员统计
D	Nina	李东礼	AI 危险区域 + 异常检测 (着火/跌倒/异常声音)

E	small-HUA	刘帅华	Django 业务 + 数据库 + Swagger
F	llppcc	刘澎湖	专职所有文档和图片绘制